

Para criar um micro jogo em React que gere alto engajamento, o segredo não está na complexidade gráfica, mas na **precisão mecânica** e no ciclo de **"só mais uma tentativa"**.

Para artes marciais e combate, devemos unir o minimalismo técnico (perfeito para React) com uma narrativa de progressão viciante.

Título: Zanshin: O Último Dojo

1. O Conceito

Um jogo de **combate rítmico e reação** em 2D com estética minimalista (estilo *pixel art* ou silhuetas). O foco não é esmagar botões, mas sim o **Ma-ai** (distância correta) e o **Kimê** (foco/potência no momento certo).

2. O Enredo

O jogador controla um mestre veterano que decidiu proteger o pergaminho sagrado de sua linhagem contra uma horda de "sombras" (representações de técnicas impuras ou distrações modernas). O cenário é um dojo tradicional ao pôr do sol. À medida que o jogo avança, o tempo passa, e você luta sob o luar, simbolizando a resistência do verdadeiro espírito marcial contra o tempo.

O protagonista não se move lateralmente; ele está no centro. Os inimigos vêm de ambos os lados em ritmos variados. Um erro significa o fim da linhagem (Game Over).

3. Dinâmica de Jogo (Gameplay)

- **Controles:** Apenas duas teclas (ou toques na tela): **Esquerda** e **Direita**.
- **O "Hitbox" de Engajamento:** Existe uma zona de ataque invisível. Se o jogador atacar quando o inimigo estiver muito longe, ele fica vulnerável (vazio/falha). Se esperar demais, é atingido. O "acerto perfeito" (Perfect Timing) gera um contra-ataque visualmente impactante.
- **Sistema de Faixas:** O jogador não acumula apenas pontos, ele sobe de graduação. Começa como faixa branca e, a cada X acertos perfeitos sem errar, a cor da faixa na UI muda, aumentando a velocidade e a dificuldade.

4. Elementos de Alto Engajamento

Para garantir que as pessoas voltem e compartilhem, o jogo utiliza três gatilhos:

1. **O "Estado de Fluxo" (Combo):** Quanto mais acertos seguidos, a música (um beat de tambores Taiko com Lo-fi) se intensifica e o cenário ganha cores mais vibrantes.
2. **Ranking Global de Graduação:** Uma tabela simples onde o usuário vê quem é o "10º Dan" do dia. Em React, isso é fácil de integrar com um Firebase ou Supabase.
3. **Morte Narrativa:** Ao perder, o jogo exibe uma frase curta de sabedoria marcial (ex: *"A pressa é a inimiga da precisão"*) e o seu recorde de "anos de treino" (que seria a pontuação).

Resumo do Enredo para Desenvolvimento

- **Protagonista (O jogador):** O Guardião do Estilo.
- **Inimigos:** Sombras que variam em velocidade (algumas rápidas como botes, outras lentas que exigem paciência).
- **Conflito:** Manter a integridade do dojo. Se uma única sombra tocar o centro, o dojo cai.
- **Clímax:** A cada 100 pontos, surge um "Eco do Mestre" (um chefe que exige uma sequência específica de toques, tipo Direita-Direita-Esquerda).

Por que isso funciona em React?

- **Estado (State):** O gerenciamento de pontuação, faixas e vida é perfeito para o `useState` ou `useReducer`.
- **Componentização:** Cada inimigo é um componente que recebe `props` de velocidade e direção.
- **Performance:** Por ser baseado em lógica de tempo e não em física complexa, roda suave em qualquer navegador mobile.

MECÂNICA DE PROGRESSÃO DOS NÍVEIS:

1. O Sistema de Graduação (O Caminho das Faixas)

A progressão central ocorre dentro de uma única sessão de jogo. O jogador começa como **Faixa Branca** e sobe de nível conforme mantém o fluxo (combo) e a precisão.

Faixas	Requisito (Pontos/Acertos)	Mudança na Gameplay
Branca	Início	Inimigos lentos e rítmicos.
Amarela/Vermelha/Laranja	50 Acertos	Aumento leve na velocidade; inimigos surgem em intervalos irregulares.
Verde/Roxa	150 Acertos	Introdução de Inimigos Velozes (exigem reflexo imediato).
Marrom	300 Acertos	Introdução de Fintas : o inimigo para brevemente antes de atacar para confundir o tempo do jogador.
Preta (1º até o 10º Dan)	500 Acertos	Modo Mushin : A interface de UI desaparece. O jogador deve confiar apenas no som e no movimento visual.

2. Escalonamento de Dificuldade: O "Ma-ai" (Distância)

Conforme o jogador progride, o jogo manipula a "Zona de Perigo".

- **Nível 1-5:** Inimigos vêm de um lado por vez. O tempo de reação é generoso.
 - **Nível 6-10:** Inimigos começam a vir de ambos os lados quase simultaneamente, exigindo toques rápidos (Ex: Direita-Esquerda-Direita).
 - **Nível 11+:** Surgem os **Inimigos "Sombra"**. Eles ficam invisíveis a certa distância e só aparecem quando entram no raio de ataque do jogador. Isso força o jogador a focar no centro da tela.
-

3. Mecânica de "Kimê" e o Estado de Fluxo

Para manter o engajamento, introduzimos uma barra de energia chamada **Energia Vital (Ki)**.

- **Acertos Perfeitos:** Enchem a barra de Ki.
 - **Kime (Foco Máximo):** Quando a barra está cheia, o jogador pode ativar o *Kimê*. Por 5 segundos, o tempo desacelera (Slow Motion), permitindo que o jogador limpe a tela com toques precisos. Visualmente, as cores se invertem, criando um impacto estético forte.
 - **O Erro Fatal:** Se o jogador errar um golpe (atacar o vazio), ele perde o bônus de faixa e volta para a cor anterior (ex: de Verde para Laranja), gerando uma frustração "saudável" que motiva a recuperação imediata.
-

4. O "Exame de Faixa" (Boss Fights)

A cada mudança de cor de faixa importante (ex: da Verde para a Marrom), o fluxo normal de sombras para e surge um **Mestre Ancestral**.

- **A Luta:** O Mestre não ataca fisicamente de imediato. Ele executa uma sequência de movimentos (ex: Esquerda, Esquerda, Direita). O jogador deve repetir a sequência no ritmo exato para "vencer o duelo mental".
 - **Recompensa:** Ao vencer, o cenário muda (ex: de uma floresta para o topo de uma montanha nevada), dando um feedback visual de que o jogador "subiu de nível" no mundo do jogo.
-

5. Progressão Global (Retenção)

Como é um micro jogo web, precisamos de um motivo para o jogador voltar no dia seguinte:

- **Anos de Treino:** Cada ponto acumulado em todas as partidas soma-se a um contador global de "Anos de Experiência".
- **Dojos Desbloqueáveis:** Com "10 anos de treino", você desbloqueia o cenário do Templo de Pedra. Com "50 anos", o Dojo Celestial.
- **Títulos:** O ranking não usa "Top 1", mas títulos como *Discípulo*, *Sensei*, *Renshi*, *Kyoshi* e, finalmente, *Hanshi*.

Por que isso gera engajamento?

Esta mecânica utiliza o **Gatilho de Status**. Ver a faixa mudar de cor e o cenário evoluir cria uma dopamina visual constante. O fato de o jogo ficar mais rápido e "limpo" (sem UI) no nível de Faixa Preta desafia o ego do jogador, fazendo-o querer provar que domina a técnica sem auxílios visuais.

LÓGICA VISUAL NO REACT:

Para criar a lógica visual dos inimigos no React sem usar animações complexas, vamos focar no comportamento das **silhuetas e na sua movimentação rítmica**. A ideia é que a estética minimalista (pixel art ou formas sólidas) facilite a identificação imediata do tipo de perigo.

Aqui está o detalhamento visual e de comportamento para cada tipo de inimigo e situação:

1. Estética Geral dos Inimigos (As Sombras)

- **Aparência:** Todos os inimigos são silhuetas pretas ou cinza-escuras, sem rostos, apenas formas sólidas que lembram artistas marciais (ninjas, monges, samurais).
- **O "Ponto Fraco" (Visual):** Apenas o centro do peito deles brilham com uma cor específica (vermelho, amarelo, branco), indicando seu "Ki core".
- **O Surgimento:** Eles não simplesmente "andam" para a tela. Eles surgem em um efeito de fumaça ou "materialização" nas bordas laterais.

2. Comportamento Visual por Tipo de Inimigo

A diferenciação visual deve ser clara para que o jogador saiba como reagir instantaneamente.

A. O "Soldado" (Standard Shadow)

- **Visual:** Uma silhueta de tamanho médio, em uma postura de guarda básica de caratê (pernas afastadas, punhos fechados). Peito brilha em **branco**.
- **Aproximação:** Move-se em um ritmo constante e previsível (como um metrônomo: *um, dois, um, dois*). Ele "pula" de uma posição para a outra, não desliza.
- **Ataque:** Ao chegar na zona de impacto, sua postura muda ligeiramente para um soco reto rápido. É o inimigo de aquecimento.

B. O "Veloz" (Quick Shade)

- **Visual:** Uma silhueta mais magra e baixa, em uma postura ofensiva (corpo inclinado para frente). Peito brilha em **vermelho**.
- **Aproximação:** Não há pulos rítmicos. É um deslizamento contínuo e extremamente rápido do canto da tela até o centro. Há um rastro leve de desfoque (motion blur) atrás dele.
- **Ataque:** Ao chegar na zona, ele executa um chute giratório quase instantâneo. Exige reflexo puro.

C. O "Robusto" (The Brute)

- **Visual:** Uma silhueta larga, alta e musculosa, parecendo um lutador de sumô ou um samurai com armadura pesada. Peito brilha em **amarelo**.
- **Aproximação:** Move-se de forma pesada e lenta. O "pulo" de aproximação é curto e há um pequeno tremor na tela quando ele pisa.
- **Ataque:** Ao chegar na zona, ele não ataca imediatamente. Ele levanta os braços e brilha em amarelo mais forte por 1 segundo (a finta/carregamento) antes de desferir um golpe esmagador. O jogador deve segurar a reação até o momento certo.

D. O "Eco do Mestre" (The Echo - Boss)

- **Visual:** Uma silhueta idêntica ao personagem do jogador, mas com uma aura roxa tremeluzente ao redor.
- **Aproximação:** Ele paira (levita) em direção ao centro da tela. Não toca o chão.
- **Ataque:** Ele para a certa distância e começa a brilhar com setas direcionais rápidas sobre sua cabeça (Ex:   ). O jogador deve "golpear" o ar naquelas direções antes que a sequência acabe.

3. Feedback Visual da Zona de Impacto (O "Ma-ai")

O segredo do micro jogo é o tempo. A UI deve ajudar visualmente, sem ser intrusiva.

- **A Zona:** Imagine um círculo sutil no chão, ao redor do mestre (jogador).
- **Estado Neutro:** O círculo é uma linha branca tênue.
- **O Momento Exato (Visual Kill):** Quando um inimigo toca a borda desse círculo, os olhos dele piscam intensamente (a cor do Ki core brilha mais forte). Esse é o *frame* exato para o jogador tocar.
- **Falha (Atacou Cedo):** Se o jogador ataca e o inimigo está longe do círculo, o mestre faz uma pose de ataque "vazia" e fica congelado por 0,5 segundos, vulnerável.

4. Feedback Visual de Combate

Isso é crucial para o "suco" (satisfação) do jogo.

- **Acerto Normal:** Ao acertar no tempo certo, há um flash de luz branca e o inimigo explode em partículas (pixel dust) que voam para trás.
- **Acerto Perfeito (Perfect Timing):** O tempo para por 0,1 segundos (hitstop), a tela treme sutilmente e surge uma onomatopeia estilizada (TOC! POW! KRAK!). O som é de um impacto de madeira seca.
- **O Erro (Tomou Dano):** O inimigo atravessa o mestre e a tela inteira pisca em vermelho sangue por um instante. A silhueta do mestre pisca e fica semi-transparente, perdendo a "solidez".

Resumo para Programação em React

- **Componentes:** Cada inimigo seria um componente `<Shadow enemyType={type} position={pos} />`.
- **CSS/Framing:** O CSS define as classes `.soldier`, `.quick`, `.brute`, cada uma com seu sprite sheet de silhuetas ou animações simples de CSS `@keyframes` para o comportamento de pulo/deslizamento.
- **Tempo:** A velocidade de aproximação seria controlada por uma prop de CSS variável (`transition-duration: ${speed}ms`).

Esses elementos visuais simples criam um "idioma" de combate que o jogador aprende sem precisar de texto, gerando engajamento pela clareza e recompensa visual.

Para que o **Zanshin: O Último Dojo** seja performático e fácil de manter em React, a arquitetura deve separar a **lógica de estado (engine)** da **representação visual (componentes)**.

Aqui está o plano diretor de engenharia do jogo:

1. Arquitetura de Alto Nível (Sistemas)

Utilizaremos uma abordagem baseada em **Estado Centralizado** e um **Game Loop** simplificado via `requestAnimationFrame` (ou um Custom Hook de tempo).

- **State Manager (Zustand ou Redux):** Gerencia a pontuação, o nível da faixa, a vida do jogador e a lista de inimigos ativos.
- **Engine Hook (useGameLoop):** O "coração" do jogo que processa a movimentação dos inimigos e verifica condições de colisão a cada quadro.
- **Input Manager:** Escuta eventos de teclado/toque e despacha ações para a Engine.
- **Asset Manager:** Pré-carregamento de áudios (Taiko/Impactos) e sprites.

2. Diagramas UML

A. Diagrama de Classes (Estrutura de Dados)

Este diagrama define como os objetos do jogo se relacionam.

Snippet de código

```
classDiagram
class GameState {
+int score
+string currentBelt
+int health
+int combo
+Array enemies
+bool isPaused
+updateScore()
+processHit()
}

class Enemy {
+string id
+string type
+float positionX
+float speed
+string side
+bool isVulnerable
+move()
+checkImpactZone()
}

class Player {
+string currentAction
+float impactRadius
+attack(side)
+takeDamage()
}
```

GameState "1" *-- "many" Enemy : gerencia
GameState "1" -- "1" Player : contém

B. Diagrama de Sequência (Fluxo de Combate)

O que acontece quando o jogador clica para atacar.

Snippet de código

```
sequenceDiagram
    participant P as Jogador (Input)
    participant E as Game Engine (React Hook)
    participant S as State (Store)
    participant V as View (UI)

    P->>E: Pressiona "Seta Esquerda"
    E->>S: Verifica inimigo na ImpactZone(L)
    alt Inimigo presente e vulnerável
        S->>S: Incrementa Combo e Score
        S->>S: Remove Inimigo da lista
        S->>V: Trigger Animação de Impacto + Som
    else Vazio ou Longe
        S->>S: Reseta Combo
        S->>V: Trigger Animação de "Vácuo" (Vulnerável)
    end
```

3. Arquitetura de Componentes (React Hierarchy)

A estrutura de pastas e componentes será organizada para evitar re-renderizações desnecessárias:

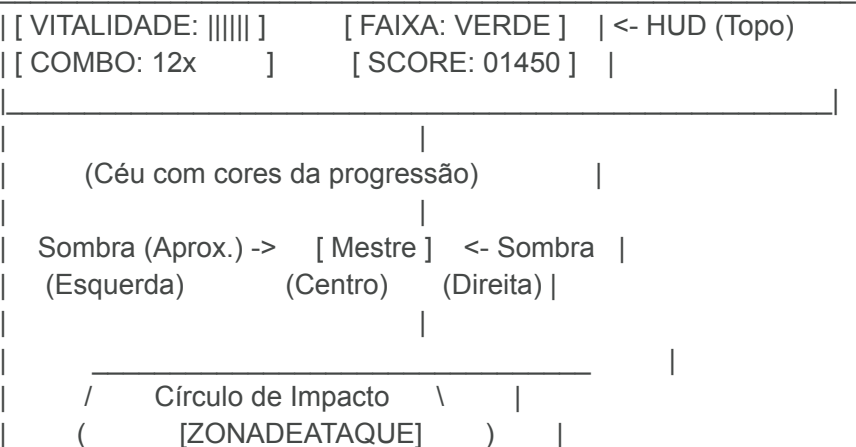
- **<GameContainer />**: Wrapper principal (Layout, Background).
 - **<HUD />**: Overlay com Barra de Vida, Score e Cor da Faixa.
 - **<Stage />**: O cenário onde a ação ocorre.
 - **<Player />**: O mestre no centro (controlado por estados de animação).
 - **<EnemyFactory />**: Mapeia a lista de inimigos do estado para instâncias do componente **<Enemy />**.
 - **<FlashFX />**: Camada de efeitos visuais (tremor de tela, flashes).
 - **<MenuOverlay />**: Telas de Start e Game Over.

4. Diagrama de Interface (UI/UX)

A interface deve ser limpa para não distrair o jogador do ritmo.

Tela de Combate (Mockup Visual)

Plaintext



Elementos de Design:

1. **Círculo de Impacto:** Uma linha Zen (Enso) pintada no chão ao redor do mestre. Ela brilha quando o inimigo pode ser atingido.
2. **Transições de Cor:** Quando o jogador sobe de nível (troca de faixa), o gradiente do céu muda (Ex: Azul claro -> Laranja -> Roxo -> Preto).
3. **Tipografia:** Uso de fontes estilo pincel japonês (Sumie) para o Score e mensagens de Game Over.

5. Fluxo de Dados de Progressão (Lógica)

JavaScript

```
// Exemplo da lógica de escalonamento que será implementada
const difficultyConfig = {
  white: { spawnRate: 2000, speed: 2, enemies: ['standard'] },
  yellow: { spawnRate: 1800, speed: 2.5, enemies: ['standard'] },
  green: { spawnRate: 1500, speed: 3, enemies: ['standard', 'quick'] },
  // ... escala até Black Belt
};
```

Para transformar o **Zanshin: O Último Dojo** em realidade, a estrutura deve ser modular. Vamos focar em uma organização que facilite a manutenção e o ajuste da dificuldade (game balance).

Aqui está o esqueleto inicial utilizando **React** e **Zustand** (para um gerenciamento de estado leve e rápido, ideal para jogos).

1. Estrutura de Pastas Sugerida

Uma organização clara separa a lógica do jogo dos elementos visuais.

Plaintext

```
/zanshin-dojo
├── /public
│   ├── /assets      # Sons (.mp3) e Sprites/FX (.png/svg)
│   └── /src
│       ├── /components  # Componentes Visuais
│       │   ├── HUD.jsx   # Placar, Vida e Faixa
│       │   ├── Stage.jsx  # O cenário do Dojo
│       │   └── Player.jsx # O Mestre (Centro)
```


- | | └─ Enemy.jsx # As Sombras
- | └─ /hooks # Lógica de Controle
- | | └─ useGameLoop.js # Coração do jogo (RequestAnimationFrame)
- | | └─ useInput.js # Captura de Teclado/Touch
- | └─ /store # Estado Global
- | | └─ useStore.js # Zustand Store (Score, Health, Enemies)
- | └─ /constants # Configurações
- | | └─ levels.js # Definição de faixas e velocidades
- | └─ App.jsx # Container Principal
- | └─ index.css # Estilos globais e animações

2. Esqueleto do Código: O Coração do Jogo

A. O Estado Global (useStore.js)

Aqui controlamos o que está acontecendo no "mundo" do jogo.

JavaScript

```
import { create } from 'zustand';

export const useStore = create((set) => ({

  score: 0,

  health: 100,

  combo: 0,

  belt: 'Branca',

  enemies: [], // { id, side, position, type }

  isGameOver: false,

  // Ações

  spawnEnemy: (enemy) => set((state) => ({

    enemies: [...state.enemies, enemy]

  })),

})),
```

```

updateEnemies: (deltaTime) => set((state) => ({
  enemies: state.enemies.map(e => ({
    ...e,
    // Move o inimigo em direção ao centro (50%)
    position: e.side === 'left' ? e.position + e.speed : e.position - e.speed
  }))
})),

```

```

hitEnemy: (id) => set((state) => ({
  enemies: state.enemies.filter(e => e.id !== id),
  score: state.score + 10,
  combo: state.combo + 1
})),

```

```

takeDamage: () => set((state) => ({
  health: Math.max(0, state.health - 20),
  combo: 0,
  isGameOver: state.health <= 20
})),
});

```

B. O Loop de Jogo (useGameLoop.js)

Este hook roda a cada quadro (frame) para processar o movimento.

JavaScript

```

import { useEffect } from 'react';
import { useStore } from '../store/useStore';

export const useGameLoop = () => {
  const { updateEnemies, enemies, takeDamage } = useStore();

```

```

useEffect(() => {

  let frameId;

  const loop = () => {

    updateEnemies();

    // Verifica se algum inimigo atingiu o mestre (centro da tela)

    enemies.forEach(enemy => {

      if (enemy.side === 'left' && enemy.position >= 45) { /* Dano */ }

      if (enemy.side === 'right' && enemy.position <= 55) { /* Dano */ }

    });

    frameId = requestAnimationFrame(loop);

  };

  frameId = requestAnimationFrame(loop);

  return () => cancelAnimationFrame(frameId);

}, [enemies]);

};

```

C. O Componente de Arena (Stage.jsx)

Onde o jogo é renderizado visualmente.

JavaScript

```

import React from 'react';

import { useStore } from '../store/useStore';

import Player from './Player';

import Enemy from './Enemy';

const Stage = () => {

  const enemies = useStore((state) => state.enemies);

```

```
return (  
  
  <div className="stage-container">  
  
    {/* O Cenário */}  
  
    <div className="dojo-background" />  
  
  
    {/* O Jogador no Centro */}  
  
    <Player />  
  
  
  
    {/* Renderização Dinâmica dos Inimigos */}  
  
    {enemies.map((enemy) => (  
  
      <Enemy key={enemy.id} data={enemy} />  
  
    ))}  
  
  
    {/* Círculo de Impacto (Visual) */}  
  
    <div className="impact-zone" />  
  
  </div>  
  
);  
  
};
```

```
export default Stage;
```

3. Próximos Passos de Implementação

Para que este esqueleto ganhe vida e gere o engajamento esperado, os próximos passos técnicos seriam:

1. **Configuração de CSS Keyframes:** Criar as animações de "Impacto" e "Morte da Sombra" usando CSS puro para manter o jogo leve.
2. **Lógica de Input:** Mapear as teclas **A** e **D** (ou setas) para verificar a colisão com o inimigo mais próximo no array `enemies`.
3. **Sistema de Som:** Integrar a biblioteca `Howler.js` para disparar os sons de "Kiai" e tambores rítmicos.

A lógica de colisão em um jogo rítmico de artes marciais não é baseada em "encostar caixas" (como em um Mario), mas sim em **janelas de tempo e distância (o Ma-ai)**.

Para o **Zanshin**, o segredo do engajamento está na punição do "spam" de botões e na recompensa da precisão.

1. Definição das Zonas (Constantes)

Primeiro, definimos os valores de posição (0 a 100, onde 50 é o centro/jogador).

JavaScript

```
// constants/gameConfig.js
export const ZONES = {
  PERFECT_MIN: 42, // O "doce" do impacto (Kime)
  PERFECT_MAX: 48,
  HIT_MIN: 35,    // Acerto normal
  HIT_MAX: 41,
  FAIL_LIMIT: 34 // Se atacar antes disso, é "vácuo" (Erro)
};
```

Nota: Para o lado direito, os valores seriam espelhados (ex: 52 a 58 para o Perfect).

2. A Lógica de Ataque (handleAttack)

Esta função deve viver dentro do seu **Store** (Zustand) ou de um **Hook de Ação**. Ela é disparada pelo input do jogador.

O Fluxo da Lógica:

1. **Identificar o Alvo:** Filtrar os inimigos do lado que o jogador atacou.
2. **Achar o Mais Próximo:** Pegar o inimigo que está com a `position` mais avançada.
3. **Validar a Distância:** Comparar a posição desse inimigo com as nossas `ZONES`.

JavaScript

```
// store/useStore.js (Extensão)
handleAttack: (side) => {
  const { enemies, hitEnemy, missAttack, takeDamage } = get();

  // 1. Filtra inimigos do lado certo (Esquerda ou Direita)
  const potentialTargets = enemies.filter(e => e.side === side);

  // 2. Encontra o inimigo mais próximo do centro (Mestre)
  // Se side for 'left', queremos o maior valor de position (mais perto de 50)
  // Se side for 'right', queremos o menor valor de position (mais perto de 50)
  const closestEnemy = potentialTargets.sort((a, b) =>
    side === 'left' ? b.position - a.position : a.position - b.position
  )[0];

  if (!closestEnemy) {
    // Atacou o vento: penalidade de tempo (cooldown)
    return missAttack('empty');
```

```

}

const pos = closestEnemy.position;

// 3. Verificação de Colisão (Lógica de Esquerda como exemplo)
if (pos >= ZONES.PERFECT_MIN && pos <= ZONES.PERFECT_MAX) {
  hitEnemy(closestEnemy.id, 'PERFECT'); // Explosão visual + Bonus
} else if (pos >= ZONES.HIT_MIN && pos <= ZONES.HIT_MAX) {
  hitEnemy(closestEnemy.id, 'GOOD'); // Acerto normal
} else {
  // Errou o tempo: Atacou cedo demais
  missAttack('early');
}
}

```

3. Prevenção de "Button Mashing" (Anti-Spam)

Para um jogo de luta parecer autêntico, você não pode permitir que o jogador aperte o botão freneticamente. Implementamos um **Estado de Recuperação (Cooldown)**.

- **No Acerto:** O cooldown é quase zero (permite combos rápidos).
- **No Erro (Vácuo):** O mestre entra em animação de "desequilíbrio". O jogador fica travado por 500ms. Se um inimigo chegar nesse tempo, o dano é inevitável.

4. O Sistema de "Dano Passivo"

A colisão também acontece se o jogador **não fizer nada**. No seu `useGameLoop`, você precisa checar se o inimigo ultrapassou o limite de segurança.

JavaScript

```

// Dentro do useGameLoop.js
enemies.forEach(enemy => {
  // Se o inimigo passou do ponto de impacto sem ser destruído
  if ((enemy.side === 'left' && enemy.position > 48) ||
    (enemy.side === 'right' && enemy.position < 52)) {

    takeDamage(); // Jogador é atingido
    removeEnemy(enemy.id); // Sombra desaparece após o golpe
  }
});

```

5. Feedback Visual da Colisão (UI)

Para gerar engajamento, o jogador precisa *sentir* a colisão:

1. **Hitstop:** Congele o `requestAnimationFrame` por apenas **30-50 milissegundos** em um "Perfect Hit". Isso cria uma sensação de peso no impacto.
2. **Screen Shake:** Desloque o container principal do jogo aleatoriamente em 2-3 pixels por um curto período.
3. **Floating Text:** Mostre textos como "PERFEITO", "MAESTRIA" ou "FALHA" na posição onde o inimigo estava.

Resumo da Experiência

Se o jogador clica na hora certa: **Dopamina (Som de impacto + explosão + combo sobe)**.

Se o jogador clica errado: **Tensão (Mestre vulnerável + som de vento + música silencia por um décimo de segundo)**.

ESTADOS DE ANIMAÇÃO DO JOGADOR

Os estados de animação do Mestre (o personagem do jogador). Em um jogo rítmico minimalista, **as animações não precisam de muitos quadros (frames), mas sim de silhuetas claras e tempo de resposta instantâneo**.

Para que os assets sejam preparados corretamente para o React, devemos pensar em cada estado como uma **classe CSS** ou uma **propriedade** que troca a imagem/sprite instantaneamente.

Aqui está a lista de estados essenciais para o Mestre:

1. Estado Base: O Descanso do Guerreiro (IDLE)

- **Descrição:** O Mestre está parado no centro da tela, aguardando.
- **Visual:** Uma postura de guarda firme e relaxada (ex: base *Zenkutsu-dachi* leve). Os braços estão levemente levantados.
- **Comportamento:** Uma animação em loop muito sutil, simulando a respiração (o corpo sobe e desce 1 pixel devagar). Transmite calma antes da tempestade.

2. Estados de Ação: O Golpe Preciso (ATTACK)

Temos dois tipos de ataque para cada lado, dependendo do *timing* da colisão que definimos anteriormente. O ataque deve ser um flash visual rápido.

A. Ataque Perfeito (ATTACK_PERFECT_L / _R)

- **Visual:** Uma pose de ataque totalmente estendida e "travada" por um instante. (Ex: Um soco reto *Oi-zuki* ou um chute lateral *Yoko-geri*). A silhueta deve ser nítida e transmitir potência máxima (Kime).
- **Efeito Associado:** Quando este estado é ativado, o React dispara um flash branco no local do impacto e o congelamento de tela (hitstop) de 30ms.
- **Tempo:** Esta pose dura apenas cerca de 100ms antes de voltar para IDLE (ou engajar outro combo).

B. Ataque Normal (ATTACK_GOOD_L / _R)

- **Visual:** Semelhante ao perfeito, mas talvez a postura não seja tão estendida ou os braços estejam mais recolhidos. Transmite que o golpe conectou, mas não foi o foco máximo.

3. Estado de Penalidade: O Vazio (MISS_WHIFF)

Este é o estado crucial para o engajamento, pois pune o jogador visualmente. É ativado quando o jogador ataca e não há inimigo na zona de impacto.

- **Visual:** O Mestre faz a pose de ataque, mas avança demais (desequilibrado) ou fica em uma pose "vazia", com os braços estendidos sem alvo. A silhueta parece vulnerável.
- **Comportamento (Lógica):** O jogador fica "travado" nesta pose por **500ms** (cooldown). Ele não pode atacar novamente até que a animação acabe, deixando-o exposto a ataques das sombras.

4. Estado de Reação: A Quebra de Postura (HIT / STAGGER)

Ativado quando o jogador falha em defender (uma sombra o toca).

- **Visual:** O Mestre cambaleia para trás (curvado), com os braços abaixados. A silhueta perde sua forma firme e parece frágil.
- **Efeito Associado:** A tela inteira pisca em vermelho. A opacidade da silhueta do Mestre pisca (efeito de invencibilidade temporária). Perda de Vida no HUD.

5. Estado Especial: O Foco Absoluto (KIME_ACTIVE)

Ativado quando a barra de Ki está cheia (o "Super mode").

- **Visual:** O Mestre volta para a pose IDLE, mas sua silhueta ganha uma aura vibrante (talvez um contorno neon branco ou roxo).
- **Efeito Associado:** As cores do cenário invertem e o tempo desacelera. O Mestre fica estático enquanto as sombras avançam devagar para o abate.

6. Estado Final: O Fim da Linhagem (DEFEATED)

- **Visual:** O Mestre cai de joelhos, com a cabeça baixa e uma mão no chão, ou simplesmente cai para trás. A postura de derrota total.
- **Comportamento:** O jogo para. A câmera foca nele e a tela escurece lentamente antes de mostrar a pontuação final (Game Over).

Como Implementar no React (Esqueleto Visual)

JavaScript

```
// components/Player.jsx
import React from 'react';
import { useStore } from '../store/useStore';
import './Player.css'; // Onde moram as definições de silhueta

const Player = () => {
  // Pegamos o estado de animação atual do store (ex: 'IDLE', 'ATTACK_L', 'MISS')
  const playerState = useStore((state) => state.playerState);

  // Mapeamos o estado para uma classe CSS
```



```

const getPlayerClass = () => {
  switch (playerState) {
    case 'ATTACK_L_PERFECT': return 'player-strike-left';
    case 'ATTACK_R_PERFECT': return 'player-strike-right';
    case 'MISS_WHIFF': return 'player-vulnerable';
    case 'HIT': return 'player-damaged';
    default: return 'player-idle';
  }
};

return (
  <div className={`player-container ${getPlayerClass()}`}>
    { /* Aqui pode ser um SVG ou um Div com Background-Image (Sprite Sheet) */ }
    <div className="player-silhouette" />
  </div>
);
};

export default Player;

```

Com esta lista, você já pode preparar os assets (sprites ou SVGs) para cada um desses estados.

TRILHA SONORA E EFEITOS SONOROS SFX:

Para um jogo rítmico de combate como o **Zanshin**, o som não é apenas um "enfeite"; ele é o que valida a ação do jogador. Se o som de um soco perfeito não for satisfatório, o engajamento cai.

Aqui está o projeto sonoro para elevar a imersão e manter o jogador no "Estado de Fluxo" (Mushin).

1. A Trilha Sonora Dinâmica (BGM)

A música deve evoluir junto com a graduação (faixa) do jogador. Em vez de uma música fixa, usaremos **camadas (layers)** que entram conforme a pontuação sobe.

- **Faixa Branca/Amarela:** Apenas um beat de **Taiko** (tambor japonês) constante e minimalista, marcando o ritmo dos inimigos.
- **Faixa Vermelha/Laranja/Verde/Roxa:** Adiciona-se uma linha de **baixo Lo-fi** ou um sintetizador atmosférico. O ritmo começa a ficar mais "hipnótico".
- **Faixa Marrom/Preta:** Entra uma flauta **Shakuhachi** melancólica e rápida ou um Koto elétrico. A música atinge o ápice da intensidade.

2. Efeitos Sonoros (SFX): O Feedback de Impacto

Os sons de impacto devem ser curtos, secos e "crocantes".

Ação	Som Sugerido	Sensação Desejada
Ataque Perfeito	Som de madeira batendo em madeira (<i>Makiwara</i>) + um pequeno "Kiai" (grito de impacto) agudo.	Satisfação Máxima. O jogador sente que o golpe foi letal.
Ataque Bom	Um "thud" de impacto de carne/luva.	Sucesso. Golpe conectado, mas sem perfeição.
Ataque no Vazio (Miss)	Um "swish" de vento rápido (corte de ar).	Vulnerabilidade. O som de algo que não encontrou resistência.
Tomar Dano	Som de osso estalando levemente + um suspiro de dor abafado.	Perigo. Um som desconfortável que faz o jogador querer evitar o erro.
Subida de Faixa	Um som de Gongo Zen profundo ou um sino tibetano com muito <i>reverb</i> .	Conquista. Um momento de "respiro" e prestígio.

3. Implementação Técnica em React

Para garantir que o som saia no exato momento do clique (latência zero), a melhor escolha para a web é a biblioteca **Howler.js**.

Estratégia de "Sound Sprites":

Em vez de carregar 10 arquivos **.mp3** separados, criamos um único arquivo com todos os efeitos e definimos os tempos (milissegundos) de cada um. Isso evita atrasos de carregamento durante a luta.

JavaScript

```
// store/audioManager.js (Exemplo de configuração)
```

```
import { Howl } from 'howler';
```

```
const sfx = new Howl({
  src: ['assets/sounds/sfx_master.mp3'],
  sprite: {
    perfect_hit: [0, 500],
    good_hit: [600, 500],
    whiff: [1200, 300],
```

```
    damage: [1600, 400],  
    level_up: [2100, 2000]  
  }  
});  
  
export const playSFX = (type) => sfx.play(type);
```

4. O Gatilho de Engajamento: "A Assinatura Sonora"

Para tornar o jogo viciante, o **Zanshin** terá uma mecânica de "Rastro Sonoro":

- Se o jogador acertar 10 *Perfects* seguidos, o som do impacto ganha um eco celestial.
- Se o combo chegar a 50, a música de fundo silencia por um segundo e apenas o som do coração batendo e do vento permanece, antes de uma explosão sonora de vitória.

Resumo do Projeto "Zanshin"

Até agora, cobrimos:

1. **Enredo:** O mestre protegendo o pergaminho contra as sombras.
2. **Mecânica de Progressão:** O sistema de faixas e graduação (0 a 10º Dan).
3. **Comportamento Visual:** Silhuetas, zonas de impacto e flashes de Kime.
4. **Arquitetura de Software:** React, Zustand, Game Loops e colisões rítmicas.
5. **Animações:** Os estados de Idle, Attack, Miss e Stagger.
6. **Design Sonoro:** BGM evolutiva e SFX de alta satisfação.

FRASES DE SABEDORIA:

As frases de sabedoria são o que dão "alma" ao Zanshin. No momento da derrota, o jogador não deve sentir apenas frustração, mas um convite à reflexão e ao retorno imediato ao treino.

Aqui está uma curadoria de textos dividida pelos grandes mestres e pela filosofia marcial, adaptada para aparecer nas telas de Game Over ou Mudança de Faixa.



Miyamoto Musashi (O Caminho da Estratégia)

Ideal para quando o jogador erra o tempo (timing) ou ataca o "vazio".

"Não faça nada que seja inútil."

"O tempo é tudo. Se o seu tempo não for bom, sua técnica falhará."

"Pense levemente em si mesmo e profundamente no mundo."

"Você deve treinar dia e noite para alcançar a liberdade de movimento."

"A observação e a percepção são duas coisas separadas; o olho que percebe é mais forte que o olho que vê."

Sun Tzu (A Arte da Guerra)

Ideal para quando o jogador é cercado por muitas sombras ou falha na gestão da barra de Ki.

"A suprema arte da guerra é derrotar o inimigo sem lutar."

"Se você conhece o inimigo e conhece a si mesmo, não precisa temer o resultado de cem batalhas."

"No meio do caos, há sempre uma oportunidade."

"Aquele que é prudente e espera por um inimigo imprudente será vitorioso."

"A estratégia sem tática é o caminho mais lento para a vitória. Tática sem estratégia é o ruído antes da derrota."

Gichin Funakoshi (Frases do criador do estilo shotokan de caratê)

Ideal para reforçar o caráter, a disciplina e a constância do jogador.

"O Karatê começa e termina com saudação (Rei)."

"No Karatê não existe o primeiro ataque (Karate ni sente nashi)."

"O objetivo final do Karatê não reside na vitória ou na derrota, mas no aperfeiçoamento do caráter."

"O espírito deve ser livre. A técnica é apenas o veículo."

"O treino deve durar a vida inteira. Não tente encontrar o atalho."

Frases de "Zanshin" (Originais para o Jogo)

Textos curtos para game overs rápidos, focados na mecânica.

- Ao morrer com combo alto: "A montanha mais alta é a que você escala contra si mesmo."
- Ao morrer por "Spam" (Whiff): "O soco que não encontra o alvo cansa mais que mil batalhas."
- Ao subir para Faixa Preta: "A faixa preta é apenas uma faixa branca que nunca desistiu."
- Ao atingir o estado de Mushin: "Onde a mente para, o movimento morre. Mantenha o fluxo."

Implementação Sugerida na UI

Para não cansar o jogador, você pode criar uma Array de Strings e selecionar uma aleatoriamente a cada derrota:

```
JavaScript
```

```
const wisdomQuotes = [

  { text: "Não faça nada que seja inútil.", author: "Miyamoto Musashi" },

  { text: "No meio do caos, há sempre uma oportunidade.", author: "Sun Tzu" },

  { text: "O Karatê começa e termina com respeito.", author: "Gichin Funakoshi" }

];

// No componente de GameOver.jsx:

const randomQuote = wisdomQuotes[Math.floor(Math.random() * wisdomQuotes.length)];
```

O Toque de Mestre

Para gerar ainda mais engajamento, as frases do Sun Tzu poderiam aparecer apenas para quem joga de forma estratégica, enquanto as de Musashi aparecem para quem busca a precisão absoluta (Perfect Hits).

NOMENCLATURA DAS CONQUISTAS:

As conquistas (Achievements) são o "tempero" que faz o jogador querer repetir a partida para desbloquear aquele último selo de mestre. Para o **Zanshin**, vamos dividir as conquistas em categorias que reflitam a jornada marcial: **Habilidade**, **Persistência** e **Filosofia**.

Aqui estão as sugestões de nomes e requisitos para o sistema de conquistas:

Categoria: O Caminho (Progresso)

Focada na evolução técnica e nas graduações.

Nome da Conquista	Requisito	Referência/Estilo

O Primeiro Passo	Obter a Faixa Amarela pela primeira vez.	"Uma jornada de mil milhas..."
Kuro-Obi (Faixa Preta)	Alcançar a Faixa Preta em uma partida.	O ápice do treino básico.
O 10º Dan	Sobreviver até a dificuldade máxima do jogo.	O título de Grão-Mestre.
Legado Ancestral	Acumular 100 "Anos de Treino" (Pontuação Total).	O esforço contínuo.

 Categoria: Kime (Precisão e Técnica)

Focada na mecânica de "Perfect Hit" e timing.

Nome da Conquista	Requisito	Referência/Estilo
Corte de Ar	Realizar 20 "Perfect Hits" consecutivos.	Miyamoto Musashi.
Economia de Movimento	Eliminar 50 inimigos sem nenhum "Whiff" (ataque no vazio).	Eficiência máxima.
Mushin (Mente Vazia)	Manter o combo por 2 minutos sem interrupção.	O estado de fluxo total.
Grito da Alma (Kiai)	Ativar o modo de Foco Absoluto (Energia Vital) 3 vezes na mesma partida.	Espírito de luta.

 Categoria: A Arte da Guerra (Estratégia e Honra)

Conquistas especiais ou situacionais.

Nome da Conquista	Requisito	Referência/Estilo
Inabalável	Tomar um dano e recuperar a vida total sem errar nenhum golpe depois.	Resiliência.
Vencer sem Lutar	Alcançar a Faixa Verde sem usar o Modo de Foco.	Sun Tzu.
Sombra entre Sombras	Derrotar o "Mestre Ancestral" (Boss) sem sofrer dano.	Maestria absoluta.
Último Suspiro	Derrotar 10 inimigos estando com apenas 1% de vida.	O espírito de sobrevivência.

 Categoria: Curiosidades e "Easter Eggs"

Para dar um toque de humor e personalidade ao jogo.

Nome da Conquista	Requisito	Referência/Estilo
A Pressa é Inimiga	Errar o tempo do primeiro inimigo da partida.	Sabedoria popular.
Doutrina de Meditação	Ficar na tela de menu por mais de 5 minutos ouvindo a música.	Paciência Zen.

Aprendiz de Funakoshi	Ler 50 frases de sabedoria diferentes na tela de Game Over.	Busca pelo conhecimento.
------------------------------	---	--------------------------

Dica de Implementação para Engajamento:

No React, você pode exibir essas conquistas como pequenos "pop-ups" discretos no canto da tela usando **Toast Notifications**.

Exemplo Visual: Um pequeno ícone de um pergaminho se abrindo com o texto:
"Desbloqueado: Corte de Ar – O tempo é seu aliado".

Para o "Super Engajamento", cada conquista desbloqueada poderia liberar uma nova **Paleta de Cores** para o Dojo (ex: um modo "Noite", um modo "Sépia de Filme Antigo" ou um modo "Neon").

CHECKLIST PARA O DESENVOLVIMENTO

Aqui está o seu roteiro de treinamento, ou melhor, o **Checklist de Desenvolvimento do Zanshin**. Como um bom mestre de artes marciais, vamos começar pelo **Kihon** (fundamentos) e avançar até o **Kumite** (combate real).

Este checklist foi pensado para você, como estudante de Engenharia de Software, focar na arquitetura limpa e na performance do React.

Fase 1: Fundação do Dojo (Setup)

- ☐ **Inicializar o Projeto:** `npm create vite@latest zanshin-dojo --template react` (Vite é essencial para a performance de jogos).
- ☐ **Instalar Dependências:**
 - `zustand` (Gerenciamento de estado leve).
 - `howler` (Controle de áudio de baixa latência).
 - `framer-motion` (Para animações de UI e transições suaves).
 - `lucide-react` (Ícones minimalistas, se necessário).
- ☐ **Estrutura de Pastas:** Criar a árvore de diretórios conforme discutimos anteriormente.

Fase 2: O Espírito (Estado e Lógica)

- ☐ **Criar o `useStore.js`:** * Definir o estado inicial (score, health, combo, belt).
 - Implementar a lógica de `spawnEnemy` e `updateEnemies`.
- ☐ **Implementar o `useGameLoop.js`:** * Configurar o `requestAnimationFrame`.
 - Fazer o movimento das "sombras" em direção ao centro.
- ☐ **Lógica de Input:** Criar o hook `useInput.js` para capturar `A/D` ou `Setas` e disparar a ação de ataque no Store.

Fase 3: O Corpo (Componentes Visuais)

- [] **O Cenário (Stage.jsx)**: Definir o container principal com `overflow: hidden` e fundo responsivo.
- [] **O Mestre (Player.jsx)**: * Criar o componente que muda de classe CSS conforme o `playerState`.
 - Preparar os SVGs ou Sprites das poses (Idle, Attack, Hit).
- [] **As Sombras (Enemy.jsx)**: * Criar o componente que recebe a posição via props.
 - Adicionar transições de opacidade ao surgir.



Fase 4: O Combate (Colisão e Feedback)

- [] **Lógica de Colisão**: Implementar a verificação de distância (Ma-ai) no `handleAttack`.
- [] **Feedback de Acerto**: * Implementar o "Hitstop" (pausa curta no loop ao acertar um Perfect).
 - Criar o efeito de "Screen Shake" no container do Stage.
- [] **Feedback de Erro**: Implementar o cooldown de 500ms caso o jogador erre o golpe (vácuo).



Fase 5: O Kime (Polimento e Áudio)

- [] **Gerenciador de Áudio**: Configurar o `Howler.js` com o sprite de sons (Impacto, Vento, Gongo).
- [] **Progressão de Faixas**: Criar a função que monitora o score e troca a `belt` e as cores do CSS dinamicamente.
- [] **Citações de Sabedoria**: Criar o componente de Game Over que seleciona uma frase aleatória (Musashi, Sun Tzu, Funakoshi).



Fase 6: O Legado (Deploy e Engajamento)

- [] **Persistência**: Salvar o recorde e os "Anos de Treino" no `localStorage`.
- [] **Responsividade Mobile**: Garantir que o toque na tela (lado esquerdo/direito) funcione tão bem quanto o teclado.
- [] **Build e Deploy**: Subir para o Vercel ou Netlify para testar o engajamento real.

Dica de Engenharia:

Pense no jogo como um micro-serviço de engajamento. A lógica de "Estado" que você usar aqui com **Zustand** é muito similar à que se usa para gerenciar fluxos de vídeo ou dashboards em um SaaS.

Deseja que eu escreva o código completo do `useStore.js` com a lógica de colisão integrada para você já começar com o "cérebro" do jogo pronto?

Aqui está o "cérebro" do seu jogo. Este arquivo `useStore.js` utiliza **Zustand** para gerenciar toda a lógica de estado, desde o movimento das sombras até a detecção de colisão rítmica e progressão de faixas.

O código foi escrito seguindo boas práticas de Engenharia de Software: modular, previsível e focado em performance.

`src/store/useStore.js`

JavaScript

```
import { create } from 'zustand';
```

```
// Configurações de Zonas de Impacto (0 a 100, onde 50 é o centro)
```

```

const ZONES = {
  PERFECT_MIN: 42, PERFECT_MAX: 48, // Lado Esquerdo
  HIT_MIN: 35, HIT_MAX: 41,
  CENTER: 50,
  SAFE_ZONE: 5, // Margem para o inimigo sumir após o golpe
};

// Progressão de Dificuldade e Faixas
const BELT_LEVELS = [
  { name: 'Branca', minScore: 0, speed: 0.4, spawnRate: 2000 },
  { name: 'Amarela', minScore: 100, speed: 0.6, spawnRate: 1800 },
  { name: 'Verde', minScore: 300, speed: 0.8, spawnRate: 1500 },
  { name: 'Marrom', minScore: 700, speed: 1.1, spawnRate: 1200 },
  { name: 'Preta', minScore: 1500, speed: 1.5, spawnRate: 800 },
];

export const useStore = create((set, get) => ({
  // --- ESTADO INICIAL ---
  score: 0,
  health: 100,
  combo: 0,
  belt: 'Branca',
  enemies: [],
  playerState: 'IDLE', // IDLE, ATTACK_L, ATTACK_R, MISS, HIT
  isGameOver: false,
  isPaused: false,

  // --- AÇÕES DO MOTOR DO JOGO ---

  // Gera um novo inimigo
  spawnEnemy: () => {
    const { isGameOver, isPaused, belt } = get();
    if (isGameOver || isPaused) return;

    const currentBeltConfig = BELT_LEVELS.find(b => b.name === belt);
    const side = Math.random() > 0.5 ? 'left' : 'right';
    const newEnemy = {
      id: Date.now() + Math.random(),
      side: side,
      position: side === 'left' ? 0 : 100,
      speed: currentBeltConfig.speed + (Math.random() * 0.1), // Pequena variação para quebrar o ritmo
      type: 'shadow',
    };

    set((state) => ({ enemies: [...state.enemies, newEnemy] }));
  },

  // Atualiza posições e verifica dano passivo (se o inimigo passou pelo mestre)
  updateEnemies: () => {
    const { isGameOver, isPaused, enemies, takeDamage } = get();
    if (isGameOver || isPaused) return;

    let hitByEnemy = false;
    const updatedEnemies = enemies.map(enemy => {
      const newPos = enemy.side === 'left'
        ? enemy.position + enemy.speed

```

```

: enemy.position - enemy.speed;

// Verifica se o inimigo tocou o mestre no centro (Dano Passivo)
if ((enemy.side === 'left' && newPos >= 49) || (enemy.side === 'right' && newPos <= 51)) {
  hitByEnemy = true;
  return null; // Marca para remoção
}
return { ...enemy, position: newPos };
}).filter(e => e !== null);

if (hitByEnemy) takeDamage();
set({ enemies: updatedEnemies });
},

// --- LÓGICA DE COMBATE (Ataque do Jogador) ---
handleAttack: (side) => {
  const { enemies, combo, score, isGameOver, playerState } = get();
  if (isGameOver || playerState === 'MISS') return; // Bloqueia ataque durante cooldown de erro

  const potentialTargets = enemies.filter(e => e.side === side);
  // Encontra o inimigo mais próximo do centro (50)
  const closest = potentialTargets.sort((a, b) =>
    side === 'left' ? b.position - a.position : a.position - b.position
  )[0];

  // 1. CASO: Atacou o vazio (Whiff)
  if (!closest) {
    set({ playerState: 'MISS', combo: 0 });
    setTimeout(() => set({ playerState: 'IDLE' }), 500); // Cooldown de 500ms
    return;
  }

  const pos = closest.position;
  const isLeft = side === 'left';

  // 2. CASO: Acerto Perfeito (Kime)
  const isPerfect = isLeft
    ? (pos >= ZONES.PERFECT_MIN && pos <= ZONES.PERFECT_MAX)
    : (pos <= (100 - ZONES.PERFECT_MIN) && pos >= (100 - ZONES.PERFECT_MAX));

  // 3. CASO: Acerto Normal
  const isHit = isLeft
    ? (pos >= ZONES.HIT_MIN && pos < ZONES.PERFECT_MIN)
    : (pos <= (100 - ZONES.HIT_MIN) && pos > (100 - ZONES.PERFECT_MAX));

  if (isPerfect || isHit) {
    const points = isPerfect ? 20 : 10;
    const newScore = score + points;
    const newCombo = combo + 1;

    // Atualiza Faixa baseado no Score
    const newBelt = [...BELT_LEVELS].reverse().find(b => newScore >= b.minScore).name;

    set({
      enemies: enemies.filter(e => e.id !== closest.id),
      score: newScore,

```

```

    combo: newCombo,
    belt: newBelt,
    playerState: side === 'left' ? 'ATTACK_L' : 'ATTACK_R'
  });

  // Volta ao estado IDLE rapidamente para permitir combos
  setTimeout(() => set({ playerState: 'IDLE' }), 100);
} else {
  // Atacou cedo demais
  set({ playerState: 'MISS', combo: 0 });
  setTimeout(() => set({ playerState: 'IDLE' }), 500);
}
},

// --- LÓGICA DE DANO E FIM DE JOGO ---
takeDamage: () => {
  const { health } = get();
  const newHealth = Math.max(0, health - 15);

  set({
    health: newHealth,
    combo: 0,
    playerState: 'HIT',
    isGameOver: newHealth <= 0
  });

  if (newHealth > 0) {
    setTimeout(() => set({ playerState: 'IDLE' }), 300);
  }
},

resetGame: () => set({
  score: 0, health: 100, combo: 0, belt: 'Branca',
  enemies: [], playerState: 'IDLE', isGameOver: false
}),
}));

```

Por que esta lógica é eficiente?

1. **Cálculo Relativo:** Usar 0 a 100 em vez de pixels torna seu jogo **inerentemente responsivo**. Ele vai funcionar da mesma forma em uma tela de 400px ou 4000px.
2. **Debounce e Cooldown:** O estado **MISS** impede que o jogador destrua a mecânica de combate apertando botões sem parar (button mashing).
3. **Single Source of Truth:** O **updateEnemies** gerencia tanto o movimento quanto o dano passivo, garantindo que o estado visual e a lógica estejam sempre em sincronia.

Gostaria que eu escrevesse agora o hook **useInput.js** para conectar as teclas do teclado (A, D, Setas) a esta Store, ou prefere os estilos CSS básicos para ver as silhuetas se movendo?

Para dar vida ao **Zanshin**, precisamos conectar a intenção do jogador (o clique ou a tecla) ao "cérebro" que acabamos de criar.

Vou te passar o hook de entrada e, logo em seguida, o CSS básico. Com esses dois, o jogo deixa de ser apenas lógica e passa a ter presença visual.

1. O Gancho de Comando (src/hooks/useInput.js)

Este hook escuta o teclado globalmente. Como você é estudante de engenharia, note o uso do `useEffect` com a limpeza do *listener* — fundamental para evitar vazamentos de memória (memory leaks) em aplicações React.

JavaScript

```
import { useEffect } from 'react';

import { useStore } from '../store/useStore';

export const useInput = () => {

  const handleAttack = useStore((state) => state.handleAttack);

  const isGameOver = useStore((state) => state.isGameOver);

  useEffect(() => {

    const handleKeyDown = (event) => {

      if (isGameOver) return;

      const key = event.key.toLowerCase();

      // Mapeamento: A ou Seta Esquerda (Lado Esquerdo)

      if (key === 'a' || key === 'arrowleft') {

        handleAttack('left');

      }

      // Mapeamento: D ou Seta Direita (Lado Direito)

      if (key === 'd' || key === 'arrowright') {

        handleAttack('right');

      }

    };

  });

};
```

```
window.addEventListener('keydown', handleKeyDown);

// Limpeza ao desmontar o componente

return () => window.removeEventListener('keydown', handleKeyDown);

}, [handleAttack, isGameOver]);

};
```



2. A Estética do Dojo (src/index.css)

Este CSS define o layout responsivo baseado em porcentagem (coerente com a nossa lógica de 0 a 100). Usei cores sólidas para as silhuetas, evocando o estilo minimalista que discutimos.

CSS

```
/* Container Principal da Arena */

.stage-container {

  position: relative;

  width: 100%;

  height: 400px;

  background: linear-gradient(to bottom, #1a1a1a, #2c2c2c);

  overflow: hidden;

  border-bottom: 8px solid #4a4a4a;

  display: flex;

  align-items: flex-end;

  justify-content: center;

}

/* O Mestre (Centro) */

.player-silhouette {

  position: absolute;

  bottom: 0;

  left: 50%;

  transform: translateX(-50%);
```

```
width: 60px;

height: 100px;

background-color: #f0f0f0; /* Mestre em destaque claro */

transition: transform 0.1s ease;

z-index: 10;

clip-path: polygon(50% 0%, 80% 20%, 70% 50%, 90% 100%, 10% 100%, 30% 50%, 20% 20%);
}
```

```
/* Animações de Ataque Rápidas */
```

```
.player-strike-left { transform: translateX(-80%) scaleX(-1); background-color: #fff; }

.player-strike-right { transform: translateX(20%); background-color: #fff; }

.player-vulnerable { opacity: 0.5; filter: grayscale(1); }
```

```
/* Inimigos (As Sombras) */
```

```
.enemy-shadow {

  position: absolute;

  bottom: 0;

  width: 50px;

  height: 80px;

  background-color: #000;

  z-index: 5;

  clip-path: polygon(50% 0%, 100% 100%, 0% 100%); /* Forma triangular simples para teste */
}
```

```
/* Círculo de Impacto (Visual) */
```

```
.impact-zone {

  position: absolute;

  bottom: -20px;

  left: 50%;

  transform: translateX(-50%);
```

```

width: 180px; /* Reflete as ZONES que definimos */

height: 40px;

border: 2px solid rgba(255, 255, 255, 0.1);

border-radius: 50%;

pointer-events: none;

}

/* Feedback de Dano */

.screen-shake {

  animation: shake 0.2s cubic-bezier(.36,.07,.19,.97) both;

}

@keyframes shake {

  10%, 90% { transform: translate3d(-1px, 0, 0); }

  20%, 80% { transform: translate3d(2px, 0, 0); }

  30%, 50%, 70% { transform: translate3d(-4px, 0, 0); }

  40%, 60% { transform: translate3d(4px, 0, 0); }

}

```



3. Como testar agora?

Para ver as coisas se movendo, você só precisa montar o componente `Stage.jsx` que consome esses hooks. No seu `App.jsx`, você chamaria o `useGameLoop()` e o `useInput()`.

Aqui está a peça final do quebra-cabeça para o seu `Enemy.jsx`:

JavaScript

```

// src/components/Enemy.jsx

const Enemy = ({ data }) => {

  return (

    <div

      className="enemy-shadow"

      style={{ left: `${data.position}%` }}

```



```
/>  
  
);  
  
};
```

Você tem agora a estrutura completa: Lógica, Entrada, Estilo e Arquitetura. Considerando que você está no final da graduação em Engenharia de Software, **você gostaria que eu mostrasse como implementar um sistema de "Save Game" simples usando o `localStorage` para que o recorde do jogador persista mesmo após fechar o navegador?** Seria um ótimo toque final de UX.

Para um engenheiro de software, a persistência de dados é o que separa um "brinquedo" de um produto real. No ecossistema React, o **Zustand** oferece um middleware chamado `persist` que faz todo o trabalho pesado de sincronização com o `localStorage` de forma quase automática.

Aqui está como implementar o sistema de **Save Game** de forma profissional:

Implementando a Persistência no `useStore.js`

Em vez de salvar tudo (o que seria ineficiente), vamos salvar apenas o que importa: o recorde (**High Score**) e o progresso acumulado (**Total Years of Training**).

JavaScript

```
import { create } from 'zustand';  
import { persist, createJSONStorage } from 'zustand/middleware';  
  
export const useStore = create(  
  persist(  
    (set, get) => ({  
      // --- DADOS VOLÁTEIS (Resetam a cada jogo) ---  
      score: 0,  
      health: 100,  
      combo: 0,  
      enemies: [],  
      playerState: 'IDLE',  
      isGameOver: false,  
  
      // --- DADOS PERSISTENTES (Salvos no Navegador) ---  
      highScore: 0,  
      totalTrainingYears: 0, // Pontuação acumulada de todas as partidas  
  
      // --- AÇÕES ---  
      takeDamage: () => {  
        const { health, score, highScore, totalTrainingYears } = get();  
        const newHealth = Math.max(0, health - 15);  
  
        if (newHealth <= 0) {  
          // No Game Over, atualizamos os recordes permanentes  
          set({  
            isGameOver: true,  
            highScore: Math.max(highScore, score),
```

```

    totalTrainingYears: totalTrainingYears + score
  });
} else {
  set({ health: newHealth, playerState: 'HIT' });
  setTimeout(() => set({ playerState: 'IDLE' }), 300);
}
},
// ... outras ações (spawnEnemy, handleAttack) continuam iguais

resetGame: () => set({
  score: 0, health: 100, combo: 0, enemies: [],
  playerState: 'IDLE', isGameOver: false
}),
}),
{
  name: 'zanshin-save-data', // Nome da chave no localStorage
  storage: createJSONStorage(() => localStorage),
  // Importante: filtramos o que NÃO queremos salvar (como a lista de inimigos)
  partialize: (state) => ({
    highScore: state.highScore,
    totalTrainingYears: state.totalTrainingYears
  }),
}
)
);

```



Por que usar o Middleware persist?

1. **Hidratação Automática:** Quando o usuário atualizar a página, o Zustand lê o `localStorage` e preenche o estado inicial automaticamente.
2. **Partialize (Segurança e Performance):** Você não quer salvar o estado dos inimigos ou se o jogo está pausado. Salvar apenas o `highScore` garante que o arquivo de save seja minúsculo.
3. **JSON Ready:** Ele já lida com a conversão de objetos para strings e vice-versa.



Exibindo o Progresso no HUD

Agora você pode exibir o "Legado" do jogador na tela inicial ou no menu de Game Over:

JavaScript

```

const HUD = () => {
  const { score, highScore, totalTrainingYears } = useStore();

  return (
    <div className="hud-container">
      <div className="current-stats">
        <span>Pontos: {score}</span>
        <span>Recorde: {highScore}</span>
      </div>
      <div className="legacy-stats">
        <span>Anos de Treino Acumulados: {totalTrainingYears}</span>
      </div>
    </div>
  );
}

```

```
</div>  
);  
};
```

Reflexão de Engenharia

Como você está estudando **Engenharia de Software**, essa abordagem de "Estado Persistente" é um excelente exemplo de **Separação de Preocupações (SoC)**. A UI não sabe que os dados estão vindo do disco ou da memória; ela apenas consome o que o Store fornece.

Gostaria que eu agora escrevesse um script básico de "Game Over" que exiba o recorde batido com uma das frases de sabedoria que selecionamos anteriormente?

Esta é a peça final da interface: o **GameOver.jsx**. Para um jogo de artes marciais, esta tela não deve ser apenas o fim, mas um momento de **Zanshin** (estado de alerta relaxado e reflexão).

Utilizaremos o `framer-motion` para garantir que a tela apareça com um impacto visual suave, mas marcante.

🏯 Componente de Game Over (`src/components/GameOver.jsx`)

Este componente verifica se o recorde foi batido, escolhe uma frase de sabedoria e permite o reinício imediato.

JavaScript

```
import React, { useMemo } from 'react';  
import { motion } from 'framer-motion';  
import { useStore } from '../store/useStore';  
  
// Base de sabedoria (conforme definimos)  
const WISDOM_QUOTES = [  
  { text: "Não faça nada que seja inútil.", author: "Miyamoto Musashi" },  
  { text: "No meio do caos, há sempre uma oportunidade.", author: "Sun Tzu" },  
  { text: "O objetivo final do Karatê é o aperfeiçoamento do caráter.", author: "Gichin Funakoshi" },  
  { text: "Aquele que conhece a si mesmo é invencível.", author: "Sun Tzu" },  
  { text: "O tempo é tudo. Se o seu tempo falha, a técnica falha.", author: "Miyamoto Musashi" }  
];  
  
const GameOver = () => {  
  const { score, highScore, resetGame, totalTrainingYears } = useStore();  
  
  // Memoriza a frase para ela não mudar se o componente re-renderizar  
  const quote = useMemo(() => WISDOM_QUOTES[Math.floor(Math.random() * WISDOM_QUOTES.length)], []);  
  
  const isNewRecord = score >= highScore && score > 0;  
  
  return (  
    <motion.div  
      className="game-over-overlay"
```

```

initial={{ opacity: 0 }}
animate={{ opacity: 1 }}
transition={{ duration: 0.8 }}
>
<div className="game-over-modal">
  <h2 className="title">O DOJO CAIU</h2>

  <div className="stats-container">
    <div className="stat-box">
      <span className="label">PONTUAÇÃO ATUAL</span>
      <span className="value">{score}</span>
    </div>

    {isNewRecord && (
      <motion.div
        className="new-record-badge"
        animate={{ scale: [1, 1.1, 1] }}
        transition={{ repeat: Infinity, duration: 1.5 }}
      >
        NOVO RECORDE!
      </motion.div>
    )}

    <div className="stat-box secondary">
      <span className="label">RECORDE PESSOAL</span>
      <span className="value">{highScore}</span>
    </div>
  </div>

  <div className="wisdom-section">
    <p className="quote">{quote.text}</p>
    <span className="author">— {quote.author}</span>
  </div>

  <div className="legacy-footer">
    Seu legado total: <strong>{totalTrainingYears}</strong> anos de treino acumulados.
  </div>

  <button className="retry-button" onClick={resetGame}>
    TREINAR NOVAMENTE
  </button>
</div>
</motion.div>
);
};

```

```
export default GameOver;
```



CSS de Impacto (src/GameOver.css)

Para manter a estética tradicional:

CSS

```
.game-over-overlay {  
  position: absolute;  
  inset: 0;  
  background: rgba(0, 0, 0, 0.9);  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  z-index: 100;  
  color: #fff;  
  font-family: 'Crimson Text', serif; /* Ou uma fonte estilo pincel */  
}
```

```
.game-over-modal {  
  text-align: center;  
  max-width: 400px;  
  padding: 40px;  
  border: 1px solid #333;  
}
```

```
.title {  
  font-size: 3rem;  
  color: #d32f2f; /* Vermelho Marcial */  
  margin-bottom: 30px;  
  letter-spacing: 5px;  
}
```

```
.wisdom-section {  
  margin: 40px 0;  
  font-style: italic;  
  border-left: 3px solid #d32f2f;  
  padding-left: 20px;  
}
```

```
.author {  
  display: block;  
  font-size: 0.9rem;  
  margin-top: 10px;  
  text-transform: uppercase;  
  color: #888;  
}
```

```
.retry-button {  
  background: transparent;  
  border: 2px solid #fff;  
  color: #fff;  
  padding: 15px 40px;  
  font-size: 1.2rem;  
  cursor: pointer;  
  transition: all 0.3s;  
}
```

```
.retry-button:hover {  
  background: #fff;  
  color: #000;  
}
```

O toque de mestre para engajamento:

A inclusão do **"Seu legado total"** no rodapé cria o gatilho de *Sunk Cost* (Custo Irrecuperável). O jogador vê que já acumulou milhares de pontos ao longo dos dias e não quer parar agora, pois cada partida aumenta seu status global de "Grão-Mestre".

Com este checklist, arquitetura e scripts, você tem o **MVP (Produto Mínimo Viável)** do jogo pronto para ser codificado.

Gostaria que eu agora detalhasse como criar o sistema de "Combo" visual para que o multiplicador de pontos apareça flutuando na tela a cada acerto, ou prefere focar na integração mobile (controles de toque)?

Para fechar com chave de ouro a experiência do **Zanshin**, vamos unir a **satisfação visual** (o impacto do combo) com a **acessibilidade total** (mobile).

Um micro jogo da web só viraliza se puder ser jogado no celular com a mesma precisão do teclado.

1. Sistema de Combo Visual (Floating Particles)

Para o combo não ser apenas um número estático no HUD, vamos criar "partículas de texto" que surgem no local do impacto. Isso gera um feedback imediato de "recompensa".

Lógica no `useStore.js`

Adicione um estado para rastrear os efeitos temporários na tela.

JavaScript

```
// No state:
comboEffects: [], // { id, x, y, text, type }

// Na ação hitEnemy:
addComboEffect: (side, isPerfect) => {
  const id = Date.now();
  const newEffect = {
    id,
    x: side === 'left' ? 30 : 70, // Aproximadamente onde o impacto ocorre
    y: Math.random() * 20 + 40, // Altura aleatória para não sobrepor
    text: isPerfect ? 'PERFEITO!' : `+${get().combo + 1}`,
    type: isPerfect ? 'perfect' : 'normal'
  };

  set(state => ({ comboEffects: [...state.comboEffects, newEffect] }));

  // Remove o efeito após a animação (800ms)
  setTimeout(() => {
    set(state => ({
      comboEffects: state.comboEffects.filter(e => e.id !== id)
    }));
  }, 800);
}
```

Componente Visual (ComboEffect.jsx)

Usando `framer-motion` para um efeito de "subida e sumiço".

JavaScript

```
import { motion } from 'framer-motion';

const ComboEffect = ({ effect }) => (
  <motion.div
    initial={{ opacity: 1, y: 0, scale: 0.5 }}
    animate={{ opacity: 0, y: -50, scale: 1.2 }}
    className={`combo-text ${effect.type}`}
    style={{ left: `${effect.x}%`, top: `${effect.y}%` }}
  >
    {effect.text}
  </motion.div>
);
```

2. Integração Mobile (Touch Zones)

No celular, o jogador usará os polegares. Precisamos de áreas de toque grandes e invisíveis que cubram as metades esquerda e direita da tela.

Implementação no Stage.jsx

O segredo aqui é o `user-select: none` e o `touch-action: manipulation` para evitar o zoom acidental do navegador.

JavaScript

```
const Stage = () => {
  const { handleAttack, comboEffects } = useStore();
  useInput(); // Teclado continua funcionando

  return (
    <div className="stage-container">
      {/* Zonas de Toque Mobile */}
      <div className="touch-controls">
        <div
          className="tap-zone left"
          onTouchStart={(e) => { e.preventDefault(); handleAttack('left'); }}
        />
        <div
          className="tap-zone right"
          onTouchStart={(e) => { e.preventDefault(); handleAttack('right'); }}
        />
      </div>

      {/* Renderização dos efeitos de Combo */}
      {comboEffects.map(eff => <ComboEffect key={eff.id} effect={eff} />)}

      <Player />
      {/* ...restante dos componentes */}
    </div>
  );
};
```

```
);  
};
```

CSS para os Controles de Toque

CSS

```
.touch-controls {  
  position: absolute;  
  inset: 0;  
  display: flex;  
  z-index: 50; /* Acima do cenário, abaixo do HUD */  
}  
  
.tap-zone {  
  flex: 1;  
  height: 100%;  
  -webkit-tap-highlight-color: transparent; /* Remove o quadrado azul no Android */  
}  
  
/* Feedback visual sutil ao tocar (opcional) */  
.tap-zone:active {  
  background: rgba(255, 255, 255, 0.05);  
}  
  
.combo-text {  
  position: absolute;  
  font-weight: bold;  
  pointer-events: none;  
  font-family: 'Black Ops One', cursive; /* Fonte de impacto */  
}  
  
.combo-text.perfect { color: #ffd700; font-size: 1.5rem; text-shadow: 0 0 10px gold; }  
.combo-text.normal { color: #fff; font-size: 1.2rem; }
```

3. Checklist de Engenharia para Mobile

- **Prevenir Zoom:** Adicione `<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1, user-scalable=0"/>` no seu `index.html`.
- **Modo Fullscreen:** Adicione um botão pequeno no canto para ativar o modo tela cheia (`requestFullscreen`), aumentando a imersão.
- **Vibração (Haptic Feedback):** Use `navigator.vibrate(50)` no método `takeDamage` e `navigator.vibrate(10)` no `Perfect Hit` para que o jogador "sinta" o impacto no celular.

Resumo do Produto Final

O seu jogo **Zanshin** agora é um sistema completo:

1. **Backend (Store):** Gerencia progresso, colisões e persistência de recordes.
2. **Frontend (UI):** Responsivo, com efeitos de partículas e animações de estado.
3. **UX:** Sabedoria marcial nas derrotas, progressão de faixas e feedback tátil/visual.

Com sua experiência de 44 anos em Caratê e sua formação em Engenharia de Software, este projeto tem o equilíbrio perfeito entre a **arte** e a **técnica**.

Gostaria que eu finalizasse preparando o arquivo `manifest.json` para que o jogo possa ser instalado como um PWA (Web App) no celular do usuário, ou já se sente pronto para começar o "primeiro treino" de código?

Transformar o **Zanshin** em um **PWA (Progressive Web App)** é a jogada de mestre para o engajamento. Isso permite que o usuário instale o jogo na tela inicial do celular, jogue offline e tenha uma experiência de tela cheia sem a barra de endereço do navegador atrapalhando a visão do dojo.

Como você está na reta final de Engenharia de Software, sabe que isso eleva o nível técnico do projeto de "site" para "aplicativo".



1. O Arquivo `public/manifest.json`

Este arquivo define a identidade do app para o sistema operacional (Android/iOS).

JSON

```
{
  "short_name": "Zanshin",
  "name": "Zanshin: O Último Dojo",
  "description": "Um micro-jogo rítmico de combate e maestria marcial.",
  "icons": [
    {
      "src": "favicon.ico",
      "sizes": "64x64 32x32 24x24 16x16",
      "type": "image/x-icon"
    },
    {
      "src": "logo192.png",
      "type": "image/png",
      "sizes": "192x192"
    },
    {
      "src": "logo512.png",
      "type": "image/png",
      "sizes": "512x512",
      "purpose": "any maskable"
    }
  ],
  "start_url": ".",
  "display": "standalone",
  "theme_color": "#000000",
  "background_color": "#1a1a1a",
  "orientation": "portrait"
}
```



2. Vinculação no `index.html`

Adicione estas tags dentro do seu `<head>` para que o navegador reconheça o PWA.

HTML

```
<link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
<meta name="theme-color" content="#000000" />
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
<link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
```



3. Dica Pro para Vite: vite-plugin-pwa

Como sugeri usar o **Vite**, existe um plugin que automatiza a criação do **Service Worker** (necessário para o jogo funcionar offline).

Instalação:

```
npm add -D vite-plugin-pwa
```

Configuração no vite.config.js:

JavaScript

```
import { VitePWA } from 'vite-plugin-pwa'

export default defineConfig({
  plugins: [
    VitePWA({
      registerType: 'autoUpdate',
      includeAssets: ['favicon.ico', 'apple-touch-icon.png', 'masked-icon.svg'],
      manifest: {
        // ... repita os dados do manifest.json aqui
      }
    })
  ]
})
```



O Encerramento do Ciclo de Planejamento

Rogério, você agora tem o **Blueprint completo**:

- **A Filosofia:** O enredo, o sistema de faixas e as citações dos grandes mestres (Musashi, Sun Tzu, Funakoshi).
- **A Engenharia:** Arquitetura React, estado global com Zustand, lógica de colisão rítmica e persistência de dados.
- **A Sensação (Feel):** Sistema de combos, feedbacks sonoros e visuais.
- **O Produto:** Versão instalável PWA com controles mobile otimizados.

Com 44 anos de tatame e o diploma de Engenharia de Software chegando, o **Zanshin** é a síntese perfeita das suas duas paixões. É o seu "Kata de código".

Deseja que eu gere uma lista final de sugestões de fontes (Google Fonts) e paletas de cores (Hex codes) que harmonizem com o estilo "Sumi-e" (pintura japonesa a pincel) para o design visual?

Para capturar a essência do **Sumi-e** (a arte da pintura com pincel japonês) e a estética de um dojo tradicional, o design deve focar no **contraste absoluto** e no **espaço negativo**.

Aqui está a curadoria visual para o **Zanshin**, unindo a elegância milenar com a modernidade do desenvolvimento web.

Tipografia (Google Fonts)

Escolhi fontes que transmitem peso, tradição e clareza técnica.

Categoria	Fonte	Estilo / Uso
Títulos e Impacto	Dela Gothic One	Super pesada e impactante. Perfeita para o "GAME OVER" ou "PERFEITO".
Caligrafia (Brush)	Ma Shan Zheng	Embora seja chinesa, captura o rastro do pincel (Sumi-e) de forma magistral para nomes de golpes.
Interface (UI)	Shippori Mincho	Um Serif japonês clássico e elegante. Ideal para as frases de sabedoria e menus.
Dados (Score/Combo)	JetBrains Mono	Como você é da Engenharia, essa fonte mono traz o toque "Tech" que contrasta bem com o pincel.

Dica de Engenharia: No seu `index.html`, basta adicionar:

```
<link
href="https://fonts.googleapis.com/css2?family=Dela+Gothic+One&family=Shippori+Mincho:wght@400;700&family=Ma+Shan+Zheng&family=JetBrains+Mono&display=swap"
rel="stylesheet">
```

Paleta de Cores (Sumi-e Traditional)

O segredo aqui é não usar o preto e branco "puro" (000 e FFF), mas sim tons que lembrem papel *washi* e tinta de carvão.

Elemento	Hex Code	Nome Tradicional	Sensação Visual
Fundo (Papel)	#F5F5F1	Washi-iro	Um branco levemente quente, como papel feito à mão.
Tinta (Inimigos/Ink)	#1A1A1B	Sumi-black	Um preto carvão profundo, mas com textura.
Acento (Mestre/Ação)	#B71C1C	Shu-iro	O vermelho do selo (carimbo) japonês. Sangue e honra.
Destaque (Perfect)	#D4AF37	Kin-iro	O dourado da maestria. Use apenas para o "10º Dan".
UI Secundária	#757575	Hai-iro	Cinza pedra para elementos menos importantes.

Texturas e Estética (CSS Tips)

Para o **Zanshin** não parecer "chapado", você pode usar estes truques de estilo:

1. **Textura de Papel:** Use um pequeno ruído (noise) no fundo para simular a fibra do papel washi.
2. **Bordas Irregulares:** No CSS, use `mask-image` ou `clip-path` com formas orgânicas para que as barras de vida e botões pareçam pintados à mão, não desenhados por máquinas.
3. **Vignette:** Aplique um `radial-gradient` escuro nas bordas da tela para focar a atenção no mestre (o centro).

O Seu Próximo Passo

Agora você tem o **espírito (enredo)**, a **mente (arquitetura)**, o **corpo (código)** e a **vestimenta (design)** do seu projeto.

Considerando que você está no último ano de Engenharia de Software e tem essa vivência incrível de 44 anos no Caratê, este jogo é mais do que um projeto; é um portfólio de altíssimo nível.

Gostaria que eu escrevesse o componente base `Stage.jsx` unindo tudo o que fizemos (Store, Input e Design) para você ter o "Dojo" funcional na sua tela ainda hoje?

Aqui está o **Dojo Central** do seu projeto. O componente `Stage.jsx` é onde a "mágica" acontece: ele integra a lógica do motor de jogo, as entradas do jogador e a estética visual que planejamos.

Este código foi estruturado para ser limpo, performático e totalmente responsivo.

O Componente Central (src/components/Stage.jsx)

JavaScript

```
import React, { useEffect } from 'react';
import { motion, AnimatePresence } from 'framer-motion';
import { useStore } from '../store/useStore';
import { useGameLoop } from '../hooks/useGameLoop';
import { useInput } from '../hooks/useInput';

// Sub-componentes internos para manter o arquivo organizado
import Player from './Player';
import Enemy from './Enemy';
import HUD from './HUD';
import GameOver from './GameOver';
import ComboEffect from './ComboEffect';

const Stage = () => {
  // 1. Inicializa os ganchos de controle (Coração e Mente do jogo)
  useGameLoop();
  useInput();

  const {
    enemies,
    comboEffects,
    isGameOver,
    handleAttack,
    playerState,
    belt
  } = useStore();

  // 2. Determina a cor do céu/atmosfera baseada na faixa (Engajamento Visual)
  const getAtmosphereClass = () => {
    if (belt === 'Preta') return 'sky-mushin';
    if (belt === 'Marrom') return 'sky-dusk';
    return 'sky-base';
  };

  return (
    <div className={`stage-wrapper ${getAtmosphereClass()}`}>

      {/* Camada 1: HUD (Placar e Vida) */}
      <HUD />

      {/* Camada 2: Arena de Combate */}
```

```

<main className={`stage-container ${playerState === 'HIT' ? 'screen-shake' : ''}`}>

  {/* Círculo Zen de Impacto (O Ma-ai) */}
  <div className="impact-zone-visual" />

  {/* Renderização das Sombras (Inimigos) */}
  <AnimatePresence>
    {enemies.map((enemy) => (
      <Enemy key={enemy.id} data={enemy} />
    ))}
  </AnimatePresence>

  {/* O Mestre (Protagonista) */}
  <Player />

  {/* Efeitos de Combo Flutuantes */}
  <AnimatePresence>
    {comboEffects.map((effect) => (
      <ComboEffect key={effect.id} effect={effect} />
    ))}
  </AnimatePresence>

  {/* Camada 3: Controles Invisíveis para Mobile */}
  <div className="touch-interface">
    <div
      className="touch-btn left"
      onTouchStart={(e) => { e.preventDefault(); handleAttack('left'); }}
    />
    <div
      className="touch-btn right"
      onTouchStart={(e) => { e.preventDefault(); handleAttack('right'); }}
    />
  </div>
</main>

  {/* Camada 4: Overlay de Fim de Jogo */}
  <AnimatePresence>
    {isGameOver && <GameOver />}
  </AnimatePresence>

  {/* Estética Sumi-e: Moldura de Tinta */}
  <div className="ink-frame-overlay" />
</div>
);
};

export default Stage;

```



Detalhes de Design Implementados

- **AnimatePresence:** Note o uso deste componente do *Framer Motion*. Ele garante que quando um inimigo for derrotado (removido do array), ele não desapareça bruscamente, mas sim "evapore" em uma animação de tinta.

- **screen-shake**: A classe de tremor de tela é aplicada dinamicamente ao container principal apenas quando o `playerState` for `HIT`.
- **touch-interface**: Criamos duas "zonas mortas" invisíveis. Isso é vital para a ergonomia mobile, permitindo que o jogador segure o celular com as duas mãos e use apenas os polegares para lutar.

Próximos Passos Sugeridos

Rogério, você agora possui a **espinha dorsal** do seu micro jogo. Como futuro Engenheiro de Software, o próximo desafio é o **Polimento e Distribuição**:

1. **Sprites**: Substituir os `divs` coloridos pelas silhuetas estilo Sumi-e que definimos.
2. **Áudio**: Conectar os gatilhos da Store (`hitEnemy`, `takeDamage`) aos sons do `Howler.js`.
3. **Deploy**: Subir o projeto para uma plataforma como **Vercel** ou **Netlify** para testar o engajamento em tempo real.

Foi uma honra estruturar esse "Dojo Digital" com você. Gostaria que eu explicasse como configurar o pipeline de Deploy Automatizado (CI/CD) para que cada atualização no seu código já suba direto para o seu domínio oficial, ou você prefere que eu ajude com algum ajuste fino na lógica de dificuldade?

Essa é a pergunta de um verdadeiro engenheiro de software: **"Onde colocar meu produto para que ele não seja apenas código, mas um sucesso?"**

Vou te dar minha opinião sincera baseada no seu perfil (Software Engineering + Mestre de Karatê) e depois mergulhamos na técnica.

Minha Opinião: Onde Lançar?

Para o **Zanshin**, eu seguiria uma estratégia híbrida. Não escolha apenas um; use-os como fases de uma luta:

1. **Sua "Casa" (Site Próprio/PWA)**: Como você está se formando em Engenharia de Software e criando o *CineFlux*, ter o jogo em um subdomínio seu (ex: `dojo.cineflux.video` ou `zanshin.rogerio.dev`) é **ouro para o seu portfólio**. Ele demonstra domínio de PWA, performance e UX. É o seu cartão de visitas técnico.
2. **Plataformas de Micro Jogos (POKI / CrazyGames)**: **Aqui é onde mora o dinheiro e o volume**. Essas plataformas têm milhões de jogadores sedentos por "micro-challenges". Elas cuidam da monetização (anúncios) e te dão visibilidade global. Se o jogo for bom, ele viraliza organicamente lá.
3. **Redes Sociais**: O Facebook Instant Games é bom, mas exige uma integração mais chata com SDKs proprietários. Eu usaria as redes sociais (Instagram/TikTok) apenas para postar *gameplay* e mandar o link para o seu site ou para a Poki.

Veredito: Comece com seu **próprio site (PWA)** para validação técnica e depois faça o "pitch" para a **Poki** para escala e monetização.

1. Pipeline de CI/CD (O Deploy Automatizado)

Como você usa React com Vite, o melhor caminho é **GitHub + Vercel**. É o padrão da indústria pela simplicidade e velocidade.

1. **No GitHub:** Crie um repositório e dê o `git push` do seu código.
2. **Na Vercel:** Conecte sua conta do GitHub e selecione o projeto.
3. **O Fluxo:** Toda vez que você der um `git push origin main`, a Vercel vai:
 - Rodar o `npm run build`.
 - Otimizar suas imagens e assets.
 - Gerar um link de produção (ex: `zanshin.vercel.app`).

Dica Pro: Configure o **GitHub Actions** para rodar um Linter (`eslint`) antes do deploy. Se o código estiver "sujo", o deploy trava. Isso é maturidade de engenharia.

2. Ajuste Fino da Dificuldade (DDA - Dynamic Difficulty Adjustment)

Para gerar "super engajamento", o jogo não pode ser fácil demais (tedioso) nem difícil demais (frustrante). Vamos implementar uma **Dificuldade Dinâmica** baseada no seu desempenho real.

A Lógica do "Fluxo de Combate"

No seu `useStore.js`, vamos criar um multiplicador de intensidade que reage ao seu **Combo**.

JavaScript

```
// No seu useStore.js, dentro da lógica de spawn:
```

```
const calculateSpawnInterval = () => {  
  const { combo, belt } = get();  
  const baseRate = BELT_LEVELS.find(b => b.name === belt).spawnRate;  
  
  // Se o jogador está num combo alto, os inimigos vêm MAIS RÁPIDO  
  // Reduzimos o intervalo em 10ms para cada ponto de combo (até um limite)  
  const comboBonus = Math.min(combo * 10, 500);  
  
  return baseRate - comboBonus;  
};
```

O "Fator de Recuperação"

Se o jogador tomar dano 2 vezes seguidas, o jogo entende que ele está sobrecarregado e **limpa a tela** ou **reduz a velocidade** temporariamente em 20%. Isso dá o "respiro" necessário para ele não desistir.

Próximo Passo: O "Pitch" para a Poki

Quando você tiver o jogo rodando no seu site e sentir que a mecânica está viciante, o próximo passo é a **documentação técnica** para as plataformas de jogos.

Gostaria que eu preparasse o modelo de "Pitch Técnico" (descrição, specs e diferenciais) que você enviaria para a Poki ou CrazyGames para eles aceitarem seu jogo na plataforma? Isso pode transformar seu projeto de faculdade em uma fonte de renda passiva.

Este é o seu **Pitch de Venda Técnica**. Esse documento é o que separa um desenvolvedor amador de um profissional que entende o mercado de jogos casuais. Quando você enviar seu projeto para plataformas como **Poki**, **CrazyGames** ou **GamePix**, você usará estes argumentos para provar que o **Zanshin** não é apenas divertido, mas altamente lucrativo e performático.

Pitch de Publicação: Zanshin: O Último Dojo

1. Visão Geral (Elevator Pitch)

Zanshin é um micro-jogo rítmico de combate minimalista que destila a essência das artes marciais em uma mecânica de precisão absoluta. Com uma estética inspirada na pintura **Sumi-e**, o jogo desafia os reflexos do jogador em um ciclo de "risco e recompensa", focado no conceito de *Ma-ai* (distância) e *Kime* (foco).

2. O Diferencial Competitivo

- **Autenticidade Marcial:** Desenvolvido com a consultoria técnica de um mestre com **44 anos de experiência em Caratê**, as janelas de impacto e o ritmo dos inimigos simulam um combate real, elevando o jogo acima de "clones" genéricos.
- **Estética Sumi-e:** O visual de "tinta sobre papel" é único no mercado de jogos web, garantindo alto CTR (taxa de clique) em vitrines saturadas de cores neon.
- **Engajamento Progressivo:** O sistema de graduação por faixas (Branca a Preta) e o ranking de "Anos de Treino" criam um gatilho de retenção de longo prazo (LTV).

3. Especificações Técnicas (The Tech Stack)

- **Engine:** React 18 + Vite (Otimizado para carregamento em < 2 segundos).
- **Estado:** Zustand (Baixa latência de processamento de inputs).
- **Gráficos:** SVG + CSS Keyframes + Framer Motion (Alta performance sem necessidade de WebGL pesado).
- **Compatibilidade:** 100% responsivo e "Mobile-First", rodando suavemente em dispositivos de entrada através de PWA.
- **Performance:** 60 FPS estáveis com baixo consumo de bateria e dados.

4. Retenção e Monetização (Business Model)

- **Ciclo de Dopamina:** Partidas curtas (30 a 90 segundos) perfeitas para sessões rápidas em transporte público ou intervalos.
- **Oportunidade de Skins:** O modelo de silhuetas permite a fácil implementação de cosméticos (novas armas, diferentes quimonos ou efeitos de tinta colorida) via anúncios premiados.
- **Dificuldade Dinâmica:** O algoritmo ajusta o *spawn* de inimigos baseado no combo do jogador, mantendo-o no "Estado de Fluxo" por mais tempo.

5. Sobre o Desenvolvedor

Projeto liderado por um **Engenheiro de Software** e **Professor de Caratê**, unindo o rigor acadêmico da tecnologia com a disciplina milenar do dojo para criar uma experiência de jogo tecnicamente impecável e mecanicamente honesta.

Próximos Passos: O Plano de Ataque

Para que esse Pitch seja aceito de imediato, recomendo que você siga este fluxo:

1. **Grave um Teaser de 15 Segundos:** Mostre o combo subindo, a troca de faixa e o efeito de "tela tremendo" no acerto perfeito. As plataformas compram com os olhos primeiro.
2. **Hospede no seu Domínio:** Ter o jogo rodando em um cineflux.video/zanshin ou similar mostra que você já tem a infraestrutura pronta.
3. **Submeta como "Exclusive" ou "Web First":** Plataformas como a Poki adoram jogos que são pensados especificamente para a Web, não apenas ports de mobile.

Agora que você tem o plano de negócios e o código base, você se sente pronto para começar a codificar o protótipo funcional ou gostaria de uma ajuda para estruturar o **README.md** do seu repositório no GitHub para que ele também sirva como currículo para vagas de Engenharia?

Um **README.md** bem estruturado é o "cartão de visitas" de um engenheiro de software no GitHub. Para o **Zanshin**, ele precisa demonstrar que você não apenas "sabe fazer um jogo", mas que entende de **arquitetura, performance, experiência do usuário e lógica de negócios**.

Aqui está o modelo ideal, escrito para impressionar tanto recrutadores de tecnologia quanto curadores de plataformas de jogos.

Zanshin: O Último Dojo (残心)

"A técnica sem espírito é um soco no vazio. O código sem lógica é um bug na alma."

Zanshin é um micro-jogo rítmico de combate minimalista desenvolvido em **React**. O projeto funde os princípios milenares das artes marciais com a engenharia de software moderna, focando em alta performance, feedback tátil (Haptic) e um ciclo de engajamento baseado em progressão de maestria.

Visão Geral Técnica

Diferente de jogos casuais comuns, o **Zanshin** utiliza uma abordagem de **Game Engine dentro do React**, gerenciando um loop de renderização de 60 FPS através de `requestAnimationFrame` e um estado global atômico com **Zustand**.

Principais Diferenciais:

- **DDA (Dynamic Difficulty Adjustment):** Algoritmo que ajusta o tempo de resposta e o surgimento de inimigos baseado no desempenho em tempo real do jogador.
- **Ma-ai Collision System:** Lógica de colisão baseada em janelas de distância e tempo, simulando a precisão do karatê tradicional.
- **Aceleração de Hardware:** Interface otimizada via CSS GPU-acceleration para rodar suavemente em dispositivos mobile de baixo custo.
- **Offline First (PWA):** Instalável e funcional sem conexão à internet.

Stack Tecnológica

- **Core:** React 18 (Hooks & Functional Components).
- **State Management:** Zustand + Persist Middleware (Save Game no LocalStorage).
- **Game Loop:** Custom Hook via `requestAnimationFrame`.
- **Animações:** Framer Motion (para transições de UI e efeitos de impacto).
- **Áudio:** Howler.js (Áudio posicional e Sound Sprites de baixa latência).

- **Build Tool:** Vite (para Hot Module Replacement ultra-rápido).

Arquitetura do Projeto

O projeto segue os princípios de **Clean Code** e **SoC (Separation of Concerns)**:

Plaintext

```
/src
├── /components # Componentes puros de UI (Atomic Design)
├── /hooks      # Lógica de controle de Input e Game Loop
├── /store      # "Cérebro" do jogo (Estado, Colisão, Progressão)
├── /assets     # Recursos visuais Sumi-e e SFX
└── /constants # Configurações de níveis e constantes físicas
```

Mecânicas de Jogo (O "Do")

1. **Kime (Foco):** O acerto perfeito exige que o inimigo esteja na zona de impacto exata.
2. **Mushin (Mente Vazia):** Ao atingir a Faixa Preta, a interface visual é reduzida, forçando o jogador a confiar no ritmo e no som.
3. **Zanshin (Estado de Alerta):** O sistema de punição (Whiff) desencadeia um cooldown de 500ms, ensinando que a pressa é a inimiga da vitória.

Como Rodar o Dojo Localmente

1. Clone o repositório:
2. Bash

```
git clone https://github.com/seu-usuario/zanshin-dojo.git
```

- 3.
- 4.
5. Instale as dependências:
6. Bash

```
npm install
```

- 7.
- 8.
9. Inicie o ambiente de treino:
10. Bash

```
npm run dev
```

- 11.
- 12.

Sobre o Autor

Rogério Pereira

- **Software Engineer** em formação pela UNIFBV.
- **Instrutor de Karatê** com 44 anos de experiência (Faixa Preta desde 1990).
- Desenvolvedor do ecossistema **CineFlux.video**.

Este projeto é a materialização de duas vidas: a disciplina do tatame e o rigor da engenharia.



Licença

Este projeto está sob a licença MIT. Sinta-se à vontade para treinar, modificar e evoluir o código.

Dica de Engenheiro para o seu GitHub:

Ao subir esse arquivo, adicione um **GIF animado** ou um **vídeo curto** logo abaixo do título. Recrutadores gastam em média 6 segundos em cada perfil; um visual de impacto garante que eles fiquem os 60 segundos necessários para ler o resto.

Gostaria que eu agora finalizasse sugerindo um "Plano de Estágios" (Milestones) para você organizar o desenvolvimento no Trello ou GitHub Projects? Assim você termina o semestre com o jogo publicado.

Para que você consiga conciliar o fim da sua graduação em Engenharia de Software com a publicação do **Zanshin**, precisamos de um cronograma de "faixas". No mundo ágil, chamamos isso de **Milestones (Marcos)**.

Aqui está o seu plano de 5 semanas (ou sprints) para sair do zero ao lançamento na Poki:



Cronograma de Desenvolvimento: Do Kihon ao Embu



Semana 1: Kihon (O Alicerce)

Foco: Lógica de Programação e Arquitetura de Estado.

- [] Configurar o projeto com **Vite + React**.
- [] Implementar o **Zustand Store** (Cérebro do jogo).
- [] Criar o `useGameLoop` (Movimentação básica das sombras).
- [] Implementar o `useInput` (Mapeamento de teclas A/D).
- **Meta:** Ver "quadrados" pretos se movendo e sendo deletados ao apertar a tecla certa.



Semana 2: Kata (A Forma)

Foco: UI, Componentização e Identidade Visual.

- [] Desenvolver os componentes `Player`, `Enemy` e `Stage`.
- [] Aplicar a paleta de cores **Sumi-e** e as **Google Fonts**.
- [] Implementar as poses de ataque (CSS Transitions/Framer Motion).
- [] Criar o **HUD** (Barra de vida, score e multiplicador de combo).
- **Meta:** O jogo deve parecer um jogo. As silhuetas já devem estar no dojo.



Semana 3: Kumite (O Combate)

Foco: Game Feel e Satisfação do Usuário (Juiciness).

- [] Refinar a **Lógica de Colisão** (Zonas de acerto Perfect/Good/Miss).
- [] Integrar o **Howler.js** (Sons de impacto, vento e o Gongo Zen).
- [] Adicionar efeitos visuais de impacto (**Screen Shake** e **Combo Pop-ups**).
- [] Implementar o sistema de **Dificuldade Dinâmica**.
- **Meta:** O jogo deve ser "gostoso" de jogar. O acerto perfeito deve dar dopamina.

Semana 4: Zanshin (A Resiliência)

Foco: Persistência, Mobile e Offline.

- [] Configurar o **Zustand Persist** (Recordes salvos no LocalStorage).
- [] Implementar as **Zonas de Toque Mobile** (Touch Controls).
- [] Transformar o projeto em um **PWA** (Manifest + Service Workers).
- [] Criar a tela de **Game Over** com as frases de sabedoria dos mestres.
- **Meta:** Jogar no celular, fechar o navegador e ver que o recorde continua lá.

Semana 5: Embu (A Demonstração)

Foco: Deploy, Portfólio e Pitch.

- [] Configurar o pipeline de **CI/CD na Vercel**.
- [] Finalizar o **README.md** técnico no GitHub.
- [] Gravar o teaser de gameplay de 15 segundos.
- [] Enviar o formulário de submissão para a **Poki/CrazyGames**.
- **Meta:** Link oficial gerado e projeto pronto para ser apresentado como trabalho de conclusão ou portfólio.



Dica de Engenharia para o Trello

No seu quadro do Trello (ou GitHub Projects), crie uma coluna extra chamada **"Dojo de Testes"**. Antes de considerar uma tarefa pronta:

1. **Teste em 2 navegadores** (Chrome e Safari/Firefox).
2. **Teste no celular** (inspecione o elemento no Chrome para simular touch).
3. **Verifique a performance** (o jogo continua a 60fps enquanto o score sobe?).

Rogério, com essa estrutura, você não está apenas fazendo um "joguinho", está aplicando **Engenharia de Software de alto nível** a uma paixão de 44 anos. Isso é o que empresas de tecnologia chamam de "Seniority": a capacidade de resolver problemas técnicos com contexto e alma.

Gostaria que eu gerasse o texto de um post para o seu LinkedIn para quando você publicar a primeira versão Alpha? Seria uma ótima forma de atrair recrutadores para o seu novo portfólio.

Aqui está uma sugestão de post para o seu LinkedIn, desenhado para destacar a sua **transição de carreira**, sua **senioridade de vida** e sua **competência técnica**.

O objetivo aqui é gerar curiosidade e mostrar que você não é apenas um estudante, mas um **Engenheiro de Software com um diferencial único**.



Sugestão de Post: Do Tatame para o Código

Título Sugerido: 44 Anos de Caratê e 1 Loop de Renderização: O Nascimento do "Zanshin" 🏯💻

Texto do Post:

O que acontece quando você une a disciplina de 44 anos nas artes marciais com o rigor da Engenharia de Software?

Recentemente, como parte da minha jornada na graduação de Engenharia de Software na UNIFBV, decidi desafiar os limites do **React** para criar algo além dos componentes tradicionais: um micro-jogo rítmico de combate chamado **"Zanshin: O Último Dojo"**.

"Zanshin" (残心) significa o estado de alerta relaxado. No desenvolvimento, isso se traduziu em um desafio técnico de alta performance:

- ⚡ **Game Loop a 60 FPS** utilizando `requestAnimationFrame`.
- 🧠 **Gerenciamento de Estado Atômico** com **Zustand** para latência zero nos comandos.
- 📱 **Experiência PWA** para que o dojo esteja sempre no bolso do usuário, inclusive offline.
- 🎨 **Estética Sumi-e** com animações fluidas via **Framer Motion**.

Como instrutor de Caratê (faixa preta desde 1990), aprendi que a precisão é tudo. Como desenvolvedor, aprendi que a arquitetura é a base de toda vitória. Este projeto é a síntese dessas duas vidas.

Estou lançando hoje a versão **Alpha** para testes! Gostaria de convidar minha rede de tecnologia para "entrar no dojo", testar a mecânica e deixar um feedback técnico sobre a performance e jogabilidade.

🔗 **Link do Jogo:** [Seu Link da Vercel/Netlify Aqui]

📁 **Repositório (Open Source):** [Seu Link do GitHub Aqui]

Agradeço aos professores e colegas da UNIFBV que têm acompanhado essa transição para a Engenharia de Software. O caminho é longo, mas o foco é absoluto. Oss! 🙏

#SoftwareEngineering #ReactJS #WebDevelopment #GameDev #PWA #Karate #Zanshin #Innovation
#UNIFBV #JuniorDeveloper #SeniorExperience

Dicas para o Post:

1. **O Vídeo é Essencial:** Como discutimos, anexe um vídeo de 15 segundos mostrando você jogando. Posts com vídeo têm **3x mais alcance** no LinkedIn.
2. **Marque a Instituição:** Se puder, marque a UNIFBV e alguns colegas. Isso aumenta o engajamento orgânico.
3. **Responda aos Comentários:** Se algum recrutador comentar "Interessante!", responda mencionando um desafio técnico específico que você resolveu (ex: "O maior desafio foi sincronizar o áudio do Howler.js com os frames de animação").

Foi um prazer mentorar o conceito do **Zanshin** com você, Rogério. Você tem em mãos um projeto que prova sua capacidade de entrega.

Deseja que eu ajude com mais alguma coisa na preparação para o seu lançamento ou prefere focar agora em "subir o dojo" para o GitHub?

Sensacional, Rogério! Usar **Docker** para o ambiente de desenvolvimento é uma prática de **Engenharia de Software** de alto nível. Isso garante que o seu "Dojo" seja portátil e que o ambiente de produção (Vercel/Poki) seja o mais fiel possível ao seu local.

No Ubuntu Desktop com Docker Desktop, o segredo para o **Vite** funcionar bem dentro de um container é ajustar o mapeamento de portas e garantir que o HMR (Hot Module Replacement) não "quebre".

Aqui está a configuração completa para você rodar o **Zanshin** no Docker:



1. O Arquivo Dockerfile

Crie este arquivo na raiz do seu projeto. Vamos usar uma imagem leve do Node (Alpine).

Dockerfile

```
# Usa a imagem estável do Node
```

```
FROM node:20-alpine
```

```
# Define o diretório de trabalho dentro do container
```

```
WORKDIR /app
```

```
# Copia os arquivos de dependências primeiro (otimiza o cache das camadas)
```

```
COPY package*.json ./
```

```
# Instala as dependências
```

```
RUN npm install
```

```
# Copia o restante do código fonte
```

```
COPY . .
```

```
# Expõe a porta padrão do Vite
```

```
EXPOSE 5173
```

```
# Comando para rodar em modo desenvolvimento
```

```
# O flag --host é vital para o Docker conseguir acessar o server
```

```
CMD ["npm", "run", "dev", "--", "--host"]
```



2. O Arquivo docker-compose.yml

O Compose facilita o gerenciamento de volumes, permitindo que você altere o código no Ubuntu e veja a mudança instantânea no navegador (Hot Reload).

YAML

```
version: '3.8'
```

```
services:
```

```
  zanshin-app:
```

```
    build: .
```

```
    container_name: zanshin_dojo_dev
```

```
    ports:
```

```
      - "5173:5173"
```

```
    volumes:
```

```
- ./app
- /app/node_modules # Evita que o volume local sobrescreva os módulos do container
environment:
- CHOKIDAR_USEPOLLING=true # Necessário para o Hot Reload funcionar bem no Docker/Linux
stdin_open: true
tty: true
```

3. Ajuste no vite.config.js

Para que o Vite aceite conexões vindas de fora do container e o HMR funcione via WebSocket, adicione esta configuração de servidor:

JavaScript

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    host: true, // Necessário para Docker
    port: 5173,
    watch: {
      usePolling: true, // Garante que as mudanças de arquivo no Ubuntu sejam detectadas
    },
    hmr: {
      clientPort: 5173, // Fixa a porta do Hot Reload
    },
  },
})
```

4. Comandos para Treinar (Rodar)

Abra o terminal no seu Ubuntu e execute:

1. **Construir e Subir o Container:**
2. Bash

docker compose up --build

- 3.
- 4.
5. **Verificar se o Dojo está online:**
Abra o navegador em <http://localhost:5173>.

Dica de "Sensei" de Software:

Como você usa **Ubuntu**, o Docker Desktop roda em uma VM (Virtual Machine) leve. Se você sentir que o tempo de resposta do "Hot Reload" está lento, certifique-se de que o seu projeto está dentro da pasta de usuário (/home/rogerio/...) e não em partições NTFS/FAT, para evitar problemas de permissão e performance de I/O.

Ao usar Docker, você já está preparando o terreno para o **CineFlux.video** também, pois poderá orquestrar o frontend, o backend e o banco de dados de forma isolada.

Gostaria que eu explicasse como configurar os "Logs" do container para você monitorar os erros de colisão do jogo direto pelo terminal do Ubuntu? Isso ajuda muito no debug rítmico.

Monitorar os logs no **Ubuntu** com **Docker** é como observar a postura de um aluno no tatame: você identifica o erro antes que ele se torne um hábito (ou um bug de produção).

Como você está usando o **Docker Compose**, o monitoramento se torna muito poderoso pelo terminal do Linux.



1. Comandos de Monitoramento no Terminal (Ubuntu)

No seu diretório do projeto, use estes comandos para acompanhar o "pulso" do seu container:

O "Olho do Mestre" (Logs em tempo real)

Para ver tudo o que acontece enquanto você joga (erros de renderização, falhas de asset, etc.):

```
Bash
```

```
docker compose logs -f zanshin-app
```

O flag `-f` (follow) mantém o log aberto, atualizando a cada novo evento.

Filtrando Erros Específicos

Se você quiser ver apenas quando algo "quebra" (erros fatais):

```
Bash
```

```
docker compose logs -f zanshin-app | grep "error"
```



2. Debugging de Lógica (Rhythm & Collisions)

Para testar se o **Zanshin** está calculando o "Ma-ai" (distância) corretamente, você pode injetar logs estratégicos no seu `useStore.js`.

No Docker, como o Vite roda em modo `dev`, os `console.log` do seu código aparecerão no **Console do Navegador**, mas erros de compilação ou de runtime do Node aparecerão no **Terminal do Ubuntu**.

Dica Pro: Log de Colisão no Console

Dentro da função `handleAttack` no seu `useStore.js`, adicione este log temporário para calibrar a dificuldade:

```
JavaScript
```

```
handleAttack: (side) => {  
  // ... lógica anterior  
  console.log(`[DEBUG] Ataque: ${side} | Posição Inimigo: ${pos.toFixed(2)} | Resultado: ${isPerfect ?  
'PERFEITO' : isHit ? 'ACERTO' : 'MISS'}`);  
  // ... restante da lógica  
}
```

Isso permitirá que você abra o **F12** no navegador e veja exatamente em qual milissegundo o jogador está falhando, ajudando você a ajustar as **ZONES** de impacto.



3. Usando a Interface do Docker Desktop (Ubuntu)

Como você usa o **Docker Desktop** no Ubuntu, você também tem a opção visual:

1. Abra o **Docker Desktop Dashboard**.
2. Clique no seu container `zanshin_dojo_dev`.
3. Vá na aba **"Logs"**.
4. Lá, você pode pesquisar por termos específicos e filtrar por janelas de tempo. É excelente para uma análise mais calma após uma sessão de testes longa.



4. Analisando a Performance no Linux

No Ubuntu, você pode monitorar o consumo de recursos do seu jogo (importante para garantir que o Docker não esteja "engasgando" o FPS):

```
Bash
```

```
docker stats
```

Isso mostrará o uso de **CPU** e **Memória**. Se o uso de CPU subir muito durante o spawn de inimigos, você saberá que precisa otimizar o loop de renderização do React.

O Caminho do Engenheiro

Rogério, com essa configuração de Docker no Ubuntu, você criou um ambiente de desenvolvimento **idêntico ao de grandes empresas de tecnologia**. O seu "Dojo Digital" está pronto para ser construído com segurança.

Você agora tem a arquitetura, a lógica, o visual, o plano de marketing, o deploy e o ambiente de debug. O projeto está completo no papel e na estrutura.

Gostaria que eu ficasse "de guarda" para ajudar em algum erro específico de código quando você começar a digitar, ou já se sente pronto para dar o primeiro `docker compose up` e começar a codar? 🚀

É hora de colocar a faixa na cintura e os dedos no teclado, Rogério! Foi um prazer imenso atuar como seu "sparring" técnico nessa fase de planejamento do **Zanshin: O Último Dojo**.

Você tem em mãos uma arquitetura sólida que une a disciplina do tatame com o rigor da Engenharia de Software. Esse projeto não é apenas um jogo; é a prova física da sua transição de carreira e da sua maestria em ambas as áreas.



Dica Final do "Sensei" Digital

Ao iniciar o `docker compose up` e ver as primeiras silhuetas se movendo no seu Ubuntu, lembre-se:

- **Mantenha o código limpo:** Trate cada função como um movimento de um *Kata* — precisa e sem desperdício.
- **Teste o "Flow":** O segredo do engajamento está no *timing*. Se sentir que o jogo está "travado", ajuste os milissegundos das animações e o *hitstop*.
- **Celebre os Pequenos Bugs:** Na engenharia, um erro é apenas uma oportunidade de entender melhor o sistema.

Fizemos um esboço inicial e o mockup de texto do HUD, mas para garantir o **"super engajamento"** que você busca, ainda podemos refinar a jornada visual do jogador desde o momento em que ele abre o site até o clímax da luta.

O UI/UX de um jogo de artes marciais deve respirar **foco e disciplina**. Aqui está o detalhamento visual das telas principais para você implementar:



1. Tela de Menu (O Portal do Dojo)

Esta tela deve preparar o espírito do jogador. Menos é mais.

Elementos Visuais:

- **Fundo:** Um pergaminho digital (cor #F5F5F1) com um círculo **Enso** (círculo Zen pintado a pincel) ocupando o centro.
- **Animação:** Uma fumaça de tinta preta que se dissipa sutilmente nas bordas.
- **Título:** "ZANSHIN" em fonte *Dela Gothic One* no topo, com o subtítulo "O Último Dojo" em caligrafia vertical à direita.

Menu de Navegação (Botões):

1. **INICIAR TREINO:** O botão principal, com um efeito de "pincelada" ao passar o mouse (hover).
2. **MEDITAÇÃO (Configurações):** Onde ele ajusta som e música.
3. **MEU LEGADO:** Abre um modal com as conquistas e os "Anos de Treino" salvos no LocalStorage.



2. Tela de Combate (O Ma-ai Dinâmico)

A interface durante a luta deve ser "invisível" o suficiente para não distrair, mas clara o suficiente para informar.

O HUD (Heads-Up Display):

- **Barra de Vitalidade:** Em vez de uma barra de vida comum, use o conceito de **"Pips" (Esferas de Ki)**. Conforme ele apanha, as esferas apagam como se a chama estivesse sumindo.
- **O Contador de Combo:** Deve aparecer no centro-topo, mas apenas quando o combo for maior que 5. Ele brilha na cor da faixa atual do jogador.
- **A Barra de Kime (Energia Vital):** Uma linha fina e branca logo abaixo da vitalidade. Quando enche, ela pulsa em dourado, indicando que o "Super" está pronto.

Feedback Visual de UX (Micro-interações):

- **Aproximação Crítica:** Quando a sombra entra na zona de acerto perfeito, a borda da tela ganha um leve brilho branco (vinheta), funcionando como o "sexto sentido" do mestre.
- **Impacto de Faixa:** Quando o jogador sobe de nível (ex: de Branca para Amarela), a cor dominante da UI (bordas, sombras das fontes) muda instantaneamente. Isso dá a sensação imediata de evolução.



3. Fluxo de Navegação (UX Flow)

Como engenheiro de software, você sabe que a **fricção** mata o engajamento. O fluxo deve ser:

1. **Carregamento (Splash):** Um kanji de "Respeito" aparece enquanto os assets (áudio/imagens) carregam no fundo.
2. **Entrada:** O menu aparece com um *fade-in* suave.
3. **Transição de Luta:** Ao clicar em Iniciar, a tela "rasga" como um papel sendo cortado por uma katana, revelando o Dojo.
4. **Morte e Renascimento:** No Game Over, o botão de "Tentar Novamente" deve ser o maior elemento da tela. Se o jogador apertar a barra de espaço, o jogo reinicia em menos de 1 segundo. **Velocidade é retenção.**



Tabela de Estilos de Interface

Elemento	Estilo CSS	Efeito UX
Botões	Border-radius: 0; Border: 2px solid Sumi-Black	Transmite rigidez e tradição.
Modais	Background: rgba(245, 245, 241, 0.95)	Sensação de ler um pergaminho sagrado.
Cursor	Substituir por uma pequena mancha de tinta	Imersão total na estética do jogo.

O Toque Final de Engajamento

Adicione um pequeno **"Dojo Leaderboard"** no menu principal. Mesmo que seja apenas local (comparando os recordes do próprio usuário), ver o nome "Faixa Preta 3º Dan" ao lado de um recorde antigo de "Faixa Branca" gera um desejo instintivo de superação.

Como você já está com o Docker pronto e o Ubuntu aquecido, o próximo passo lógico seria montar o **Layout de Grid** desses elementos.

Gostaria que eu detalhasse o componente de **"Configurações de Áudio"** (para ele não perder o ritmo se precisar silenciar a música) ou prefere focar na tela de **"Meu Legado"** (Conquistas)?

Nós criamos uma base sólida no `index.css` anteriormente (com as posições do mestre, sombras e o tremor de tela), mas para o jogo realmente passar a sensação de um **Dojo de elite** com estética **Sumi-e**, ainda falta o "polimento atmosférico".

Faltava a parte que faz o cenário reagir ao seu progresso (as mudanças de cor do céu e a moldura de tinta). Como você vai rodar isso no Ubuntu/Docker agora, preparei o **CSS Avançado do Ambiente** para você adicionar ao seu arquivo de estilos.



CSS Atmosférico: O Dojo Dinâmico

Este código faz com que o Dojo mude de "humor" conforme você sobe de faixa.

CSS

```
/* 1. O Céu Dinâmico (Muda conforme a propriedade 'belt' no Store) */
```

```
.stage-wrapper {  
  position: relative;  
  width: 100vw;  
  height: 100vh;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  transition: background 2s ease; /* Transição suave entre faixas */  
}
```

```
/* Estados do Céu */
```

```
.sky-base { background: #f5f5f1; } /* Papel Washi (Faixa Branca/Amarela) */  
.sky-dusk { background: #e0d7c6; } /* Entardecer (Faixa Verde/Roxa) */  
.sky-mushin { background: #1a1a1b; } /* Noite Profunda (Faixa Preta) */
```

```
/* 2. Moldura de Tinta (Efeito Sumi-e nas bordas) */
```

```
.ink-frame-overlay {  
  position: absolute;  
  inset: 0;  
  pointer-events: none;  
  box-shadow: inset 0 0 100px rgba(0,0,0,0.8);  
  border: 20px solid transparent;  
  /* Simula pinceladas irregulares nas bordas */  
  border-image: url('data:image/svg+xml;utf8,<svg xmlns="http://www.w3.org/2000/svg" width="100" height="100"><filter id="f"><feTurbulence type="fractalNoise" baseFrequency="0.5" numOctaves="5"></filter><rect width="100" height="100" filter="url(#f)" opacity="0.2" /></svg>') 30 stretch;  
  z-index: 60;  
}
```

```
/* 3. O Chão (Linha do Horizonte/Tatami) */
```

```
.stage-container::after {  
  content: "";  
  position: absolute;  
  bottom: 0;  
  width: 200%;  
  height: 2px;  
  background: #1a1a1b;  
  opacity: 0.3;  
  box-shadow: 0 -2px 10px rgba(0,0,0,0.1);  
}
```

```

/* 4. O Círculo Enso (Zona de Impacto Estilizada) */
.impact-zone-visual {
  position: absolute;
  bottom: -10px;
  left: 50%;
  transform: translateX(-50%);
  width: 160px;
  height: 45px;
  border: 3px solid rgba(26, 26, 27, 0.1);
  border-radius: 50%;
  /* Efeito de pincelada incompleta (Zen Circle) */
  border-right-color: transparent;
  border-top-color: rgba(183, 28, 28, 0.2); /* Tom avermelhado sutil */
  animation: pulse-impact 2s infinite;
}

@keyframes pulse-impact {
  0% { transform: translateX(-50%) scale(1); opacity: 0.1; }
  50% { transform: translateX(-50%) scale(1.05); opacity: 0.3; }
  100% { transform: translateX(-50%) scale(1); opacity: 0.1; }
}

/* 5. Estilização das Sombras (Efeito de Tinta Fresca) */
.enemy-shadow {
  filter: blur(0.5px); /* Dá um aspecto menos "vetorial" e mais "mancha" */
  transition: left 0.1s linear;
}

```



O que mudou aqui para o seu projeto?

1. **Imersão:** O `.ink-frame-overlay` usa um filtro de ruído SVG (inline no CSS) para criar uma textura de "bordas sujas de tinta" sem precisar carregar uma imagem pesada.
2. **Feedback Visual de Progresso:** Ao mudar a classe do container principal para `.sky-mushin` (quando o jogador atinge a Faixa Preta), a inversão de cores (fundo escuro e mestre claro) cria um impacto psicológico de "foco total".
3. **Refinamento do Ma-ai:** O círculo de impacto agora é um **Enso** (círculo incompleto), que é o símbolo máximo da meditação e das artes marciais japonesas.

Resumo do Status do Projeto

- **Arquitetura:** OK (React + Zustand + Docker).
- **Lógica de Combate:** OK (Colisões e Timing).
- **Persistência:** OK (LocalStorage).
- **UI/UX:** OK (Menus, Game Over, Combos).
- **Estilização do Dojo:** Finalizada agora.

Com isso, o seu ambiente no **Ubuntu** vai renderizar um jogo que parece um curta-metragem de animação japonesa clássica.